NUMPY, PANDAS, MATPLOTLIB

Numpy :-

What is Numpy :-

- Numpy stands for numerical python. It is the core and extremely popular library for scientific computations.
- Numpy contains large number of mathematical algebraic functions. Numpy is a Linear Algebraic Library for python.
- Numpy array essentially come in two flavours : vectors and matrices. Vectors are strictly 1-d arrays and matrices are 2-d and 3-d.

How to install NumPy :-

- Open the terminal or command prompt and execute the following command :- pip install numpy
- To use NumPy in a Python program, import it as follows :- import numpy as np

- **Examples :-**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr)  // Output :- [10 20 30 40 50]
```

Types of NumPy arrays :-

1. One-dimensional Array (1D)

- A one-dimensional array (1D array) in NumPy is a collection of elements arranged in a single sequence or line.
- It stores multiple values in one dimension and allows us to access each element using its index.

- **Examples :-** import numpy as np

```
arr = np.array([10, 20, 30, 40, 50])

print(arr)    //  Output :- [10 20 30 40 50]

print(arr[0])   // Output :- 10
```

2. Two-dimensional Array (2D)

- A two-dimensional array (2D array) in NumPy is a collection of elements arranged in rows and columns, similar to a mathematical matrix or a table.
- It is used to store and process data in two dimensions.

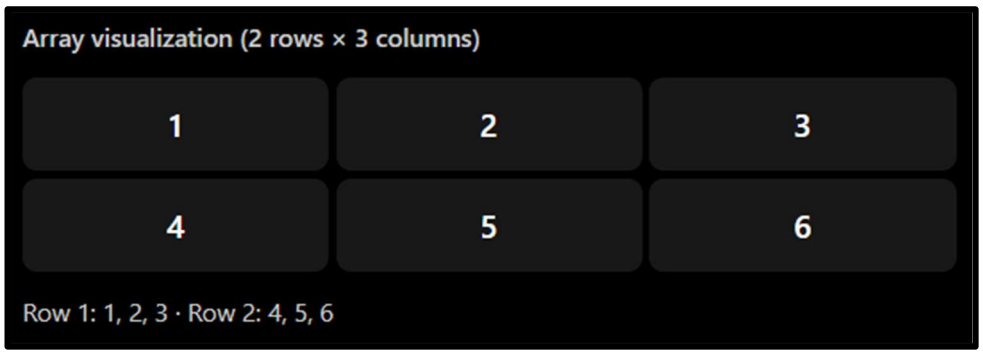
- **Examples :-** import numpy as np

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)   //  Output :- [[1, 2, 3], [4, 5, 6]]

print(arr[0, 1])    // Output :- 2

print(arr[1, 2])    // Output :- 6
```



3. Multidimensional Array (3D)

- A multidimensional array in NumPy is an array that contains data in three or more dimensions.
- It is used to represent complex data that cannot be conveniently stored in a simple 1D or 2D array.

◆ **Examples :-**

```
import numpy as np

arr = np.array([  [[1, 2], [3, 4]],  [[5, 6], [7, 8]] ])

print(arr)  // Output :- [  [[1, 2], [3, 4]],  [[5, 6], [7, 8]] ]
```

Different Ways for Creating Array in Numpy :-

1. np.arange( )

- A np.arange() is used to create an array containing a sequence of numbers within a specified range.
- **Syntax :-** np.arange(start, stop, step)

◆ **Examples :-** import numpy as np

```
arr = np.arange(1, 10, 2)

print(arr)  // Output :- [1 3 5 7 9]
```

Another Example :-

```
arr = np.arange(5)

print(arr)  // Output :- [0 1 2 3 4]
```

2. np.zeros( )

- A np.zeros() is used to create an array in which all elements are initialized to zero.
- Syntax :- np.zeros(shape)

◆ **Examples :-** import numpy as np

```
arr = np.zeros((2, 3))

print(arr)    // Output :- [[0. 0. 0.]  [0. 0. 0.]]
```

3. np.ones( )

- A np.ones() is used to create an array in which all elements are initialized to one.

◆ **Examples :-** import numpy as np

```
arr = np.ones((2, 3)) // Output :- [[1. 1. 1.] [1. 1. 1.]]
```

4. np.random.randint( )

- A np.random.randint() is used to create an array containing random integer values within a specified range.
- Syntax :- np.random.randint(low, high, size)

◆ **Examples :-** import numpy as np

```
arr = np.random.randint(1, 10, 5)

print(arr)  //  Output :- [3 8 1 6 4]
```

Creating a 2D Array :-

```
arr = np.random.randint(1, 10, size=(2, 3))

print(arr)  // Output :- [[4 7 2] [9 1 6]]
```

5. np.eye( )

- np.eye( ) is used to create a 2D array with 1s on the main diagonal and 0s everywhere else. It is commonly used to create an identity matrix.

Syntax :- np.eye(N, M=None, k=0)

- **N** ⟶ Number of rows.
- **M** ⟶ Number of columns. It is optional.
- **k** ⟶ Position of the diagonal. The default value is 0, which represents the main diagonal.

◆ **Examples 1 :- Creating an Identity Matrix**

```
import numpy as np

arr = np.eye(3)

print(arr) // Output :- [[1. 0. 0.]  [0. 1. 0.]  [0. 0. 1.]]
```

◆ **Examples 2 :- Using Rows and Columns**

```
import numpy as np

arr = np.eye(2, 4)

print(arr)   // Output :- [[1. 0. 0. 0.]  [0. 1. 0. 0.]]
```

◆ **Examples 3 :- Changing the Diagonal Using k**

```
import numpy as np

arr = np.eye(3, k=1)

print(arr)  // Output :- [[0. 1. 0.]  [0. 0. 1.]  [0. 0. 0.]]
```

NumPy Array Attributes :-

- A NumPy array attributes are properties that provide information about an array, such as its dimensions, shape, size, and data type.

Important NumPy Attributes :-

1. ndim :- A ndim returns the number of dimensions of a NumPy array.

2. shape :- A shape returns the number of elements along each dimension of the array.

3. size :- A size returns the total number of elements present in the array.

4. dtype :- A dtype returns the data type of the elements stored in the array.

5. itemsize :- A itemsize returns the number of bytes used to store one element of the array.

6. nbytes :- A nbytes returns the total number of bytes occupied by all elements of the array.

- **Examples :-** import numpy as np

```
arr = np.array([[10, 20, 30], [40, 50, 60]])

print("Dimensions :", arr.ndim)   // Output :- Dimensions : 2

print("Shape :", arr.shape)    // Output :- Shape : (2, 3)

print("Size :", arr.size)     // Output :- Size : 6

print("Data type :", arr.dtype)    // Output :- Data type : int64

print("Item size :", arr.itemsize)  // Output :- Item size : 8

print("Total bytes :", arr.nbytes)  // Output :- Total bytes : 48
```

NumPy Vs Python List :-

Python List	Numpy
Inside List we have elements of multiple data type.	All the elements must be of same data type.
List are slow compared to numpy.	Numpy arrays are faster than lists.
We can't do element wise operation.	We can do element wise operations in numpy
Takes more size.	Takes less size as compared to list.

- **Examples :-** Import numpy as np

```
list1 = [10,20,30]

list2 = [20,30,40]

print(list1 + list2)  // Output :- [10,20,30,20,30,40]
```

```
arr1 = np.array([10,20,30])

arr2 = np.array([20,30,40])

print(arr1 + arr2 ) // Output :- [30,50,70]
```

Reshaping Numpy Arrays :-

- The reshape() function in NumPy is used to change the shape or dimensions of an array without changing its data or the total number of elements.
- Syntax :- array.reshape(rows, columns)

Example 1 :- 1D Array to 2D Array

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

new_arr = arr.reshape(2, 3)

print(new_arr)  // Output :- [ [1 2 3]  [4 5 6] ]
```

Example 2 :- Changing 2D Array Shape

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

new_arr = arr.reshape(3, 2)

print(new_arr)   // Output :- [[1 2]  [3 4]  [5 6]]
```

Different Parameter Reshape Example :- (Syntax :- np.reshape(a, newshape, order = 'C'))

```
Import numpy as np

a = np.arrange(12)

print(a)   // Output :- [0,1,2,3,4,5,6,7,8,9,10,11]

c = np.reshape(a,(4,3))

print(c)   // Output :- ( [ [0,1,2] , [3,4,5] , [6,7,8], [9,10,11] ] )

x = np.arrange(12).reshape((4,3))

print(x)   // Output :- ( [ [0,1,2] , [3,4,5] , [6,7,8], [9,10,11] ] )

arr1 = np.reshape(a, (4,3), order='C')        // C = Row Wise And F = Column Wise

print(arr1)   // Output :- ( [ [0,1,2] , [3,4,5] , [6,7,8], [9,10,11] ] )

arr2 = np.reshape(a, (4,3), order='F')

print(arr2)   // Output :- ( [ [0,4,8] , [1,5,9] , [2,6,10], [3,7,11] ] )
```

Numpy reshape() with unknown Dimension :-

- We can give an unknown dimension in reshape function. Means that we do not have to specify an exact number for one of the dimensions in the reshape ( ) function.
- For that unknown dimension, we have to write "-1" as the value. Numpy will calculate this number for you.

Example :- Import numpy as np

a = np.arrange(10).reshape((5,2))

print(a) // Output :- [[0,1], [2,3], [4,5], [6,7], [8,9]]

a.reshape((2,-1))   // Output :- [[0,1,2,3,4], [5,6,7,8,9]]

a.reshape((5,-1)) // Output :- [[0,1], [2,3], [4,5], [6,7], [8,9]]

a.reshape((-1,5)) // Output :- [[0,1,2,3,4], [5,6,7,8,9]]

Reshape(-1,1) Example :-

x = np.arrange(10)

print(x) // Output :- [0,1,2,3,4,5,6,7,8,9]

a.reshape((-1,1))   // Output :- [[0],[1],[2],[3],[4],[5],[6],[7],[8],[9]]

Numpy.reshape( ) Vs Array.resize( ):-

- np.resize( ) function is used to create a new array with the specified shape. Array.resize( ) function is used to create a new array with the specified shape.
- np.resize( ) not change the original array. Array.resize( ) change the original array.

Numpy.resize( ) Example :-

Import numpy as np

x = np.arrange(10)

v = np.resize(x, (5,3))

print(v) // output :- [[0,1,2], [3,4,5], [6,7,8], [9,0,1], [2,3,4]]

Array.resize( ) Example :-

Import numpy as np

x = np.arrange(10)

v = x.resize((5,3))

print(v) // Output :- [[0,1,2], [3,4,5], [6,7,8], [9,0,0], [0,0,0]]

Aliasing And Cloning Numpy Array :-

- **Aliasing :-** The process of giving another reference variable to the existing numpy array is called aliasing.
- In aliasing, both variables point to the same original array, so changing one variable also changes the other.

Example :-

```
import numpy as np

arr1 = np.array([10, 20, 30])

arr2 = arr1

arr2[0] = 100

print("arr1 :", arr1)   // Output :- arr1 : [100  20  30]

print("arr2 :", arr2)   // Output :- arr2 : [100  20  30]
```

- **Cloning :-** Cloning means creating a separate copy of an existing NumPy array so that changes made to the copied array do not affect the original array.
- We can implement cloning using a copy( ) function.

Example :-
```
import numpy as np

arr1 = np.array([10, 20, 30])

arr2 = arr1.copy()

arr2[0] = 100

print("arr1 :", arr1)      // Output :- arr1 : [10 20 30]

print("arr2 :", arr2)     // Output :- arr2 : [100  20  30]
```

Ndarray.base Attribute in NumPy :-

- The base attribute of a NumPy ndarray is used to identify the original array that owns the data when the current array is a view of another array.
- It returns the base array if the array shares its data with another array; otherwise, it returns None.

Example 1 : Using base

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])

arr2 = arr1[1:3]

print(arr2)   // Output :- [20 30]

print(arr2.base)    // Output :- [10 20 30 40]
```

Example 2 : base with copy()

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])

arr2 = arr1.copy()

print(arr2.base)   // Output :- None
```

Note :- copy() creates an independent array, so arr2 does not have arr1 as its base.

View Vs Copy in NumPy :-

- A view is a new NumPy array that shares the same underlying data with the original array. Therefore, changes made to the view can also affect the original array.
- A copy is a completely independent NumPy array that has its own data. Therefore, changes made to the copy do not affect the original array.

View Example :-

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])

arr2 = arr1.view()

arr2[0] = 100

print("Original :", arr1)   // Original : [100  20  30  40]

print("View :", arr2)   // View : [100  20  30  40]
```

Copy Example :-

```
import numpy as np

arr1 = np.array([10, 20, 30, 40])

arr2 = arr1.copy()

arr2[0] = 100

print("Original :", arr1)   // Original : [10 20 30 40]

print("Copy :", arr2)   // Copy : [100  20  30  40]
```

Axis in NumPy :-

- An axis in NumPy represents a particular dimension of an array along which an operation is performed.
- In a 2D array, axis 0 represents the rows direction (downward), while axis 1 represents the columns direction (across).

Example :-

```
import numpy as np

marks = np.array([ [80, 70, 90], [60, 75, 85], [90, 80, 95] ])

print("Marks :", marks)      // Output :- Marks : [[80 70 90]  [60 75 85]  [90 80 95]]

print("Total of each subject :", np.sum(marks, axis=0))    // Output :- Total of each subject : [230 225 270]

print("Total of each student :", np.sum(marks, axis=1))    // Output :- Total of each student : [240 220 265]
```

Easy Way to Remember :-

axis=0 ⟶ Down ↓ ⟶ Column-wise

axis=1 ⟶ Across ⟶ Row-wise

Sum in NumPy :-

- The sum() function in NumPy is used to calculate the sum of all elements of an array or the sum of elements along a specific axis.
- Syntax :- np.sum(array, axis=None)
- array ⟶ The NumPy array on which the operation is performed.
- axis ⟶ Specifies the direction of the operation. It is optional.

Example :- import numpy as np

```
arr = np.array([[10, 20, 30], [40, 50, 60]])

print("Total sum:", np.sum(arr))  // Output :- Total sum : 210

print("Column-wise sum:", np.sum(arr, axis=0))   // Output :- Column-wise sum : [50 70 90]

print("Row-wise sum:", np.sum(arr, axis=1))   // Output :- Row-wise sum : [60 150]
```

Product and Sort in Numpy :-

- The np.prod() function is used to calculate the product of all elements in a NumPy array or the product of elements along a specified axis.
- Syntax :- np.prod(array, axis=None)

Example 1 :- Product of All Elements

```
import numpy as np

arr = np.array([2, 3, 4, 5])

result = np.prod(arr)

print(result)   // Output :- 120
```

Calculation :- 2x3x4x5 = 120

Example 2 :- Product in a 2D Array

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(np.prod(arr))  // Output :- 720

print(np.prod(arr, axis=0))  // Output :- [4 10 18]

print(np.prod(arr, axis=1))   // Output :- [6 120]
```

np.sort() – Sorting NumPy Arrays

- The np.sort() function is used to arrange the elements of a NumPy array in ascending order by default.

Example 1 :- Sorting a 1D Array

```
import numpy as np

arr = np.array([50, 20, 40, 10, 30])
```

```
sorted_arr = np.sort(arr)

print(sorted_arr)  // Output :- [10 20 30 40 50]
```

Example 2 :- Sorting a 2D Array

```
import numpy as np

arr = np.array([[30, 10, 20], [60, 40, 50]])

print(np.sort(arr))    // Output :- [[10 20 30]  [40 50 60]]
```

Vertical Stacking (vstack) in NumPy :-

- The np.vstack() is a NumPy function used to combine two or more arrays vertically, one array on top of another.
- Syntax :- np.vstack((array1, array2))

Example :-

```
import numpy as np

arr1 = np.array([[1, 2, 3], [4, 5, 6]])

arr2 = np.array([[7, 8, 9], [10, 11, 12]])

result = np.vstack((arr1, arr2))

print(result)   // Output :- [[ 1  2  3] [ 4  5  6] [ 7  8  9] [10 11 12]]
```

Horizontal Stacking (hstack) in NumPy :-

- The np.hstack() is a NumPy function used to combine two or more arrays horizontally, side by side.
- Syntax :- np.hstack((array1, array2))

Example :-

```
import numpy as np

arr1 = np.array([[1, 2],[3, 4]])

arr2 = np.array([[5, 6],[7, 8]])

result = np.hstack((arr1, arr2))

print(result)  // Output :- [[1 2 5 6] [3 4 7 8]]
```

Splitting Arrays in NumPy :-

- The np.split() is a NumPy function used to divide an array into multiple smaller arrays at specified positions or into a specified number of equal parts.
- Syntax :- np.split(array, sections)

Example 1 :- Split into Equal Parts

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

result = np.split(arr, 3)

print(result)   // Output :- [array([10, 20]), array([30, 40]), array([50, 60])]
```

Example 2 :- Store Split Arrays in Variables

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

a, b, c = np.split(arr, 3)

print("A :", a)    // Output :- A : [1 2]

print("B :", b)    // Output :- B : [3 4]

print("C :", c)    // Output :- C : [5 6]
```

Splitting Arrays (Split function in Numpy 2D) :-

- The np.split() is used to divide a 2D NumPy array into multiple smaller arrays along a specified axis.
- It can split the array row-wise using axis=0 or column-wise using axis=1.
- Syntax :- np.split(array, sections, axis)

Example :-

```
import numpy as np

arr = np.array([[10, 20, 30, 40], [50, 60, 70, 80], [90, 100, 110, 120],[130, 140, 150, 160]])

print("Original Array :" , arr)   // Original Array : [[ 10  20  30  40] [ 50  60  70  80] [ 90 100 110 120] [130 140 150 160]]

print("\nSplit Row-wise :")

print(np.split(arr, 2, axis=0))    // Split Row-wise : [array([[10, 20, 30, 40], [50, 60, 70, 80]]), array([[ 90, 100, 110, 120], [130, 140, 150, 160]])]

print("\nSplit Column-wise :")

print(np.split(arr, 2, axis=1))   // Split Column-wise : [array([[ 10,  20], [ 50,  60], [ 90, 100], [130, 140]]), array([[ 30,  40], [ 70,  80], [110, 120], [150, 160]])]
```

Arithmetic Operations in Numpy :-

- NumPy provides built-in functions to perform arithmetic operations on arrays.
- These functions can perform calculations element-wise, meaning the operation is performed between corresponding elements of arrays.

Common Arithmetic Functions :-

Function	Operation	Example
np.add()	Addition	np.add(a, b)
np.subtract()	Subtraction	np.subtract(a, b)
np.multiply()	Multiplication	np.multiply(a, b)
np.divide()	Division	np.divide(a, b)
np.power()	Power	np.power(a, b)
np.mod()	Remainder	np.mod(a, b)

Example :-

```
import numpy as np

a = np.array([10, 20, 30, 40])

b = np.array([2, 4, 5, 8])

print("Addition :", np.add(a, b))    // Addition : [12 24 35 48]

print("Subtraction :", np.subtract(a, b))   // Subtraction : [ 8 16 25 32]

print("Multiplication :", np.multiply(a, b))  // Multiplication : [ 20  80 150 320]

print("Division :", np.divide(a, b))  // Division : [5. 5. 6. 5.]

print("Power :", np.power(a, b))   // Power : [      100       160000     24300000  6553600000000000]

print("Remainder :", np.mod(a, b))  // Remainder : [0 0 0 0]
```

Note : The power values can become very large because np.power(a, b) calculates :-

$10^2 = 100$

$20^4 = 160000$

$30^5 = 24300000$

$40^8 = 65536000000000$

Slicing Operations in Numpy :-

- Slicing in NumPy is the process of extracting a specific portion of an array by using a range of index positions.
- It allows us to select one or more elements from an array without accessing every element individually.
- Syntax :- array[start:stop:step]

Example :- Slicing a 1D Array

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])

print(arr[1:4])    // Output :- [20 30 40]

print(arr[:3])   // Output :- [10 20 30]

print(arr[3:])   // Output :- [40 50 60]

print(arr[::2])  // Output :- [10 30 50]

print(arr[::-1])  // Output :- [60 50 40 30 20 10]
```

Indexing and Slicing in Numpy (2D Array) :-

- A 2D NumPy array contains elements arranged in rows and columns.
- Indexing is used to access individual elements, while slicing is used to extract a portion of the array.
- Indexing Syntax :- array[row_index, column_index] and For Slicing :- array[row_start:row_end, column_start:column_end]

Example :-

```
import numpy as np

marks = np.array([ [80, 75, 90], [65, 85, 70], [90, 95, 88], [70, 80, 75] ])

print(marks)   // Output :- [[80 75 90] [65 85 70] [90 95 88] [70 80 75]]

print(marks[1])  // Output :- [65 85 70]

print(marks[0:2, 0:2])  // Output :- [[80 75] [65 85]]

print(marks[::-1])  // Output :- [[70 80 75] [90 95 88] [65 85 70] [80 75 90]]

print(marks[:, 0:2])  // Output :- [[80 75] [65 85] [90 95] [70 80]]
```

Broadcasting in Numpy :-

- The term broadcasting describes how numpy treats arrays with different shapes during arithmetic operations.
- Numpy arrays differ from a normal python list because of their ability to broadcast.
- However, operations on arrays of non-similar shapes is still possible in numpy, because of the broadcasting capability.
- Broadcasting allows NumPy to perform calculations between arrays without manually changing their shapes.
- If the dimensions of two arrays are dissimilar, element-to-element operations are not possible.

Example 1 :-

```
import numpy as np

marks = np.array([ [70, 80, 90], [60, 75, 85], [80, 90, 95] ])

grace = np.array([5, 5, 5])

result = marks + grace

print(result)   // Output :- [[ 75  85  95] [ 65  80  90] [ 85  95 100]]
```

Example 2 :-

```
import numpy as np

 arr = np.array([ [10, 20, 30], [40, 50, 60] ])

result = arr * 2

print(result)  // Output :- [[ 20  40  60] [ 80 100 120]]
```

How to Convert a Pandas DataFrame to a Numpy Array :-

```
import numpy as np

import pandas as pd

data = pd.read_csv('iris.csv')

data.head()

data_numpy = data.values

print(data_numpy)

print(data.columns)

data1 = data['SepalLengthCm'].values

print(data1)
```

Concatenate in Numpy :-

- numpy.concatenate() is a NumPy function used to join two or more arrays into a single array along an existing axis.
- Syntax :- np.concatenate((array1, array2), axis=0)

Example :- Joining 1D Arrays

```
import numpy as np

arr1 = np.array([10, 20, 30])

arr2 = np.array([40, 50, 60])

result = np.concatenate((arr1, arr2))

print(result)   // Output :- [10 20 30 40 50 60]
```

Example :- Joining 2D Arrays Row-wise

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

result = np.concatenate((arr1, arr2), axis=0)

print(result)   // Output :- [[1 2] [3 4]  [5 6]  [7 8]]
```

Example :- Joining 2D Arrays Column-wise

```
import numpy as np

arr1 = np.array([[1, 2],[3, 4]])

arr2 = np.array([[5, 6],[7, 8]])

result = np.concatenate((arr1, arr2), axis=1)

print(result)   // Output :- [[1 2 5 6]  [3 4 7 8]]
```

Conditional Selection in Numpy :-

```
Import numpy as np

a = np.arrange(11,20)

print(a)   // Output :- [11,12,13,14,15,16,17,18,19]
```

Q1. Display all the numbers greater than 15 from the given numpy array.

```
X = a > 15                                  Or          x = np.where( a > 15 )

Print(X)   // Output :- [False, False, False, False, False, True ,True ,True ,True]

Print(a[x])  // Output :- [16,17,18,19]
```

Q2. Display all even numbers from the given numpy array.

Print(a) // Output :- [11,12,13,14,15,16,17,18,19]

X = a % 2 = = 0 Or x = np.where(a % 2 == 0)

Print(X) // Output :- [False, True, False, True, False, True, False, True, False]

a [x] // Output :- [12,14,16,18]

Q3. Display the common elements from the given two numpy arrays.

a = np.array([1,1,8,8,3,8,9,6,9,8])

b = np.array([8,1,7,8,3,3,4,9,9,1])

x = a == b Or x = np.where(a == b)

print(x) // Output :- [False, True, False, True, True, False, False, False, True, False]

a[x] // Output :- [1,8,3,9]

Q4. Display the Numbers which are greater than 5 but less than 10.

a = np.array([1,1,8,8,3,8,9,6,9,8])

x = (a>5)&(a<10) Or x = np.where(a > 5) & (a < 10)

print(a[x]) // Output :- [8,8,8,9,6,9,8]

Seed Function in Numpy :-

- np.random( ) is used to generate random numbers. If we use np.random( ) gives totally unpredictable responses.
- To make np.random( ) predictable we need pass seed value using np.random.seed(value).
- np.random.seed(value) function is used to save the state of a random function.
- So that it can generate same random numbers on multiple executions on the same machine or on different machines.
- With Seed Syntax :- np.random.seed(value)
- Without Seed Syntax :- np.random.randint(low, high, size)

Example :-

import numpy as np

np.random.seed(42)

numbers = np.random.randint(1, 11, 5)

print(numbers) // Output :- [7 4 8 5 7]

Mean, Median & Mode in Numpy :-

1. Mean

- Mean is the average value of a dataset, calculated by dividing the sum of all values by the total number of values.
- Formula :-

$$\text{Mean} = \frac{\sum_{i=1}^{n} x_i}{n}$$

- x_i = Each value in the dataset
- n = Total number of values
- $\sum x_i$ = Sum of all values

Example :-

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

print("Mean :", np.mean(data))   // Mean : 30.0
```

2. Median

- Median is the middle value of a dataset when the values are arranged in ascending or descending order.
- Formula :- If the number of observations n is odd :-

$$Median = x_{\frac{n+1}{2}}$$

- Formula :- If the number of observations n is even :-

$$Median = \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}$$

Example :-

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

print("Median :", np.median(data))     // Median : 30.0
```

3. Mode

- Mode is the value that occurs most frequently in a dataset.
- NumPy does not provide a direct np.mode() function, so we can use np.unique() with return_counts=True.

Example :-

```
import numpy as np

data = np.array([10, 20, 20, 30, 20, 40, 30])
```

```
values, counts = np.unique(data, return_counts=True)

mode = values[np.argmax(counts)]

print("Mode :", mode)    // Mode : 20
```

Loading and Saving NumPy Arrays – I/O with NumPy :-

- I/O (Input/Output) in NumPy refers to reading NumPy arrays from files and saving NumPy arrays to files on disk.
- NumPy mainly provides save(), load(), savetxt(), and loadtxt() for this purpose.

1. Saving NumPy Array to a File

- The np.save() function is used to save a NumPy array into a binary .npy file.
- Syntax :- np.save(filename, array)

Example :-

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

np.save("data.npy", arr)      // data.np file The array is stored in this file. ( Binary File )
```

2. Loading NumPy Array from a File

- The np.load() function is used to load an array from a .npy file.
- Syntax :- np.load(filename)

Example :-

```
import numpy as np

arr = np.load("data.npy")

print(arr)    // Output :- [10 20 30 40 50]
```

3. Saving Multiple NumPy Arrays

- The np.savez() is used to save multiple NumPy arrays into a single .npz file.
- Each array can be given a name, which can later be used to access that specific array.

Example :-

```
import numpy as np

names = np.array(["Amit", "Rahul", "Neha"])

marks = np.array([85, 90, 78])

ages = np.array([20, 21, 20])

np.savez("student_data.npz", names=names, marks=marks, ages=ages)    // Output :- student_data.npz
```

```
# Load the file

data = np.load("student_data.npz")

# Access individual arrays

print("Names:", data["names"])    // Output :- Names : ['Amit' 'Rahul' 'Neha']

print("Marks:", data["marks"])    // Output :- Marks : [85 90 78]

print("Ages:", data["ages"])     // Output :- Ages : [20 21 20]
```

4. Saving Array as a Text File

- The np.savetxt() function is used to save a NumPy array in a text file such as .txt or .csv.

Example :-

```
import numpy as np

arr = np.array([[10, 20, 30], [40, 50, 60]])

np.savetxt("data.txt", arr)        // Output :- data.txt ( Text File )
```

5. Loading Array from a Text File

- The np.loadtxt() function is used to load numerical data from a text file.

Example :-

```
import numpy as np

arr = np.loadtxt("data.txt")

print(arr)     // Output :- [[10. 20. 30.]  [40. 50. 60.]]
```

6. Saving and Load Csv File

```
import numpy as np

arr = np.array([11, 12, 13, 14, 15])

np.savetxt("outfile.csv", arr, delimiter = ' , ')        // Output :- outfile.csv ( Csv File )

b = np.loadtxt('outfile.csv')

print(b)  // Output :- [11,12,13,14,15]
```

Argmax and Argmin in Numpy :-

- The np.argmax() returns the index position of the maximum value in a NumPy array, while np.argmin() returns the index position of the minimum value.
- Syntax argmax :- np.argmax(array)   and Syntax argmin :- np.argmin(array)

np.argmax( ) Example :-

```
import numpy as np

arr = np.array([10, 25, 5, 40, 15])

result = np.argmax(arr)

print("Index of maximum value :", result)     // Output :- Index of maximum value : 3

print("Maximum value :", arr[result])     // Output :- Maximum value : 40
```

np.argmin( ) Example :-

```
import numpy as np

arr = np.array([10, 25, 5, 40, 15])

result = np.argmin(arr)

print("Index of minimum value :", result)   // Output :- Index of minimum value : 2

print("Minimum value :", arr[result])   // Output :- Minimum value : 5
```

argmax() and argmin() in 2D Array Using Axis :-

```
import numpy as np

arr = np.array([ [10, 50, 30], [80, 20, 60], [40, 90, 70] ])

print("Maximum index in each column :", np.argmax(arr, axis=0))     // Output :- Maximum index in each column : [1 2 2]

print("Maximum index in each row :", np.argmax(arr, axis=1))    // Output :- Maximum index in each row : [1 0 1]
```

Pandas :-

Introduction To Pandas :-

- Pandas is an open-source Python library that is used for data manipulation, data analysis, data cleaning, and processing of structured data.

Why Do We Need to Learn Pandas?

- Pandas makes it easy to store, organize, and manipulate large amounts of structured data.
- Pandas allows us to read data from different sources such as CSV files, Excel files, JSON files, and databases.
- Pandas provides functions for filtering, sorting, selecting, and modifying data according to our requirements.
- Pandas allows us to easily identify, remove, and replace missing values in a dataset.

Pandas Data Structures :-

- Pandas uses different data structures to store, organize, and manipulate data efficiently. The main traditional data structures are Series, DataFrame.

1. Series

- A Series is a one-dimensional labeled data structure used to store a sequence of data values.
- A Series can store numerical values, strings, or other Python data types.
- Each value in a Series is associated with an index label that identifies its position.
- A Series can be created using the pd.Series() function.

Example :-

```
import pandas as pd

marks = pd.Series([80, 75, 90, 85])

print(marks)
```

Output :-

```
0    80
1    75
2    90
3    85
dtype: int64
```

2. DataFrame

- A DataFrame is a two-dimensional labeled data structure that represents data in the form of rows and columns.
- A DataFrame can contain multiple columns, and each column can have a different data type.
- Each row and column in a DataFrame can be identified using labels or indexes.
- A DataFrame is the most commonly used Pandas data structure for data analysis and data manipulation.
- A DataFrame can be created using the pd.DataFrame() function.

Example :-

```
import pandas as pd

data = {

   "Name" : ["Amit", "Rahul", "Neha"],

   "Marks" : [80, 75, 90]

}

df = pd.DataFrame(data)

print(df)
```

Output :-

```
   Name  Marks

0  Amit    80

1 Rahul    75

2  Neha    90
```

Different Ways to Create a Series :-

- A Series can be created from different Python and NumPy data structures such as List, Tuple, Dictionary, NumPy Array, and Scalar Value.

Creating Series Using a List :-

```
import pandas as pd

data = [10, 20, 30, 40]

series = pd.Series(data)

print(series)
```

Output :-

```
0   10

1   20

2   30

3   40

dtype: int64
```

Creating Series Using a Tuple :-

```
import pandas as pd

data = (10, 20, 30, 40)

series = pd.Series(data)

print(series)
```

Creating Series Using a Dictionary :-

```
import pandas as pd

data = { "Math": 80, "Science": 90, "English": 75 }

series = pd.Series(data)

print(series)
```

Output :-

```
Math      80

Science   90

English   75

dtype: int64
```

Note :-

Dictionary Key ⟶ Series Index

Dictionary Value ⟶ Series Value

Creating Series Using a Scalar Value :-

```
import pandas as pd

data = [10, 20, 30, 40]

series = pd.Series(data, index=["A", "B", "C", "D"])

print(series)
```

Pandas Series [Customized Indexes & datatype, Access Elements, Operations] :-

- A Pandas Series is a one-dimensional labeled data structure that stores values along with their corresponding index labels.
- By default, Pandas assigns integer indexes, but we can customize the indexes and specify the datatype of the Series.

Customized Indexes :-

- Customized indexes are user-defined labels that are assigned to the elements of a Pandas Series instead of using the default integer indexes.

Example :-

```
import pandas as pd

marks = pd.Series( [80, 90, 75], index=["Math", "Science", "English"] )

print(marks)
```

Output :-

```
Math      80
Science   90
English   75
dtype: int64
```

Specifying Datatype :-

- The dtype parameter is used to specify the data type of the values stored in a Pandas Series.

Example :-

```
import pandas as pd

marks = pd.Series(  [80, 90, 75], dtype="float" )

print(marks)

print(marks.dtype)
```

Output :-

```
0    80.0
1    90.0
2    75.0
dtype: float64
float64
```

Customized Index and Datatype Together :-

```
import pandas as pd

marks = pd.Series( [80, 90, 75], index=["Math", "Science", "English"], dtype="float64" )

print(marks)

print(marks.dtype)
```

Accessing Elements of a Series :-

- We can access Series elements using their index labels or integer positions.

Example :-

```
import pandas as pd

marks = pd.Series([11, 22, 33, 44, 55])

# Display the Series

print(marks)
```

Output :-

```
0   11
1   22
2   33
3   44
4   55
```

A. Access Using Index Label

print(marks[1]) // Output :- 22

B. Access Using Position :- We can use .iloc[ ] to access an element based on its integer position.

print(marks.iloc[1]) // Output :- 22

C. Access Multiple Elements :-

print(marks.iloc[0:2])

Output :-

0 11

1 22

Operations on Pandas Series :- Pandas allows us to perform mathematical operations directly on Series.

Example :-

```
import pandas as pd

a = pd.Series([10, 20, 30])

b = pd.Series([5, 10, 15])

print(a + b)
```

Addition Output :-

0 15

1 30

2 45

dtype: int64

Other Arithmetic Operations

```
print(a - b)   # Subtraction

print(a * b)   # Multiplication

print(a / b)   # Division
```

Subtraction Output :-

0 5

1 10

2 15

Multiplication Output :-

0 50

1 200

2 450

Division Output :-

0 2.0

1 2.0

2 2.0

Pandas DataFrame :-

- A Pandas DataFrame is a two-dimensional labeled data structure that stores data in the form of rows and columns, similar to a table or an Excel spreadsheet.
- A DataFrame can contain multiple rows and columns.

Creating DataFrame Using Dictionary

- A DataFrame can be created from a dictionary in which dictionary keys become column names and dictionary values become column data.

Example :-

```
import pandas as pd
```

Output :-

	Name	Marks
0	Amit	80
1	Rahul	90
2	Neha	75

```
data = {
   "Name": ["Amit", "Rahul", "Neha"],
   "Marks": [80, 90, 75]
}
df = pd.DataFrame(data, index=["0", "1", "2"])
print(df)
```

Creating DataFrame Using List of Lists

- A DataFrame can be created from a list of lists, where each inner list represents one row of the DataFrame.

Example :-

```
import pandas as pd
data = [ ["Amit", 80], ["Rahul", 90], ["Neha", 75] ]
df = pd.DataFrame(data, columns=["Name", "Marks"])
print(df)
```

Output :-

```
   Name  Marks
0  Amit    80
1 Rahul    90
2  Neha    75
```

Creating DataFrame Using List of Dictionaries

- A DataFrame can be created from a list of dictionaries, where each dictionary represents one row and its keys represent column names.

Example :-

```
import pandas as pd
data = [ {"Name": "Amit", "Marks": 80}, {"Name": "Rahul", "Marks": 90}, {"Name": "Neha", "Marks": 75} ]
df = pd.DataFrame(data)
print(df)
```

Creating DataFrame Using Pandas Series

- A DataFrame can be created by combining multiple Pandas Series into a dictionary.

Example :-

```
import pandas as pd
names = pd.Series(["Amit", "Rahul", "Neha"])
marks = pd.Series([80, 90, 75])
df = pd.DataFrame({
   "Name": names,
   "Marks": marks
})
print(df)
```

Creating DataFrame Using NumPy Array

- A DataFrame can be created from a NumPy array by passing the array to the pd.DataFrame() function.

Example :-

```
import pandas as pd

import numpy as np

data = np.array([ [10, 20], [30, 40], [50, 60] ])

df = pd.DataFrame( data, columns=["A", "B"], index={1,2,3} )

print(df)
```

Output :-

```
   A  B
1 10  20
2 30  40
3 50  60
```

Viewing / Inspecting Data in Pandas DataFrame :-

- Viewing and inspecting data in Pandas means examining the contents, structure, size, column names, data types, and basic information of a DataFrame.
- Pandas provides several functions that help us quickly understand a dataset before performing data analysis.

Example :-

```
import pandas as pd

data = {
   "Name": ["Amit", "Rahul", "Neha", "Priya", "Raj"],
   "Age": [20, 21, 19, 22, 20],
   "Marks": [85, 90, 78, 95, 88]
}

df = pd.DataFrame(data)

print(df)
```

Output :-

```
   Name  Age  Marks
0  Amit   20    85
1 Rahul   21    90
2  Neha   19    78
3 Priya   22    95
4   Raj   20    88
```

1. head

- The head() function is used to view the first five rows of a DataFrame by default.
- Example :- print(df.head())
- We can specify the number of rows :- print(df.head(3))

Output :-

```
   Name  Age  Marks
0  Amit   20    85
1 Rahul   21    90
2  Neha   19    78
```

2. tail

- The tail() function is used to view the last five rows of a DataFrame by default.
- Example :- print(df.tail())
- We can also specify the number of rows :- print(df.tail(2))

Output :-

```
   Name  Age  Marks
3 Priya   22    95
4   Raj   20    88
```

3. shape

- The shape attribute returns the number of rows and columns in the DataFrame.
- Example :- print(df.shape)

4. Columns

- The columns attribute is used to display the column names of the DataFrame.
- Example :- print(df.columns)

5. index

- The index attribute is used to display the row index labels of the DataFrame.
- Example :- print(df.index)

6. dtypes

- The dtypes attribute displays the data type of each column in the DataFrame.
- Example :- print(df.dtypes)

7. Info

- The info() function provides a summary of the DataFrame, including the number of rows, columns, non-null values, and data types.
- Example :- df.info()

8. describe

- The describe() function provides statistical information about numerical columns in the DataFrame.
- Example :- print(df.describe())

9. Size

- The size attribute returns the total number of elements present in the DataFrame.
- Example :- print(df.size)
- Output :- 15 Because, 5 rows x 3 columns = 15

Example :-

```
import pandas as pd

# Create a DataFrame

data = {

   "Name": ["Amit", "Rahul", "Neha", "Priya", "Raj"],

   "Age": [20, 21, 19, 22, 20],

   "Marks": [85, 90, 78, 95, 88]

}
```

```
df = pd.DataFrame(data)

# Display the first three rows

print("First 3 Rows :")

print(df.head(3))

# Display the last two rows

print("\nLast 2 Rows :")

print(df.tail(2))

# Display the number of rows and columns

print("\nShape :")

print(df.shape)

# Display column names

print("\nColumns :")

print(df.columns)

# Display row indexes

print("\nIndex :")

print(df.index)

# Display data types

print("\nData Types :")

print(df.dtypes)

# Display DataFrame information

print("\nDataFrame Information :")

df.info()

# Display statistical summary

print("\nStatistical Summary :")

print(df.describe())

# Display total number of elements

print("\nSize :")

print(df.size)
```

```
First 3 Rows:
    Name  Age  Marks
0   Amit   20     85
1  Rahul   21     90
2   Neha   19     78

Last 2 Rows:
    Name  Age  Marks
3  Priya   22     95
4    Raj   20     88

Shape:
(5, 3)

Columns:
Index(['Name', 'Age', 'Marks'], dtype='str')

Index:
RangeIndex(start=0, stop=5, step=1)
```

```
Data Types:
Name       str
Age      int64
Marks    int64
dtype: object

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    5 non-null      str
 1   Age     5 non-null      int64
 2   Marks   5 non-null      int64
dtypes: int64(2), str(1)
memory usage: 252.0 bytes
```

```
Statistical Summary:
             Age       Marks
count   5.000000    5.000000
mean   20.400000   87.200000
std     1.140175    6.300794
min    19.000000   78.000000
25%    20.000000   85.000000
50%    20.000000   88.000000
75%    21.000000   90.000000
max    22.000000   95.000000

Size:
15

Dimensions:
2
```

Pandas DataFrame Column Selection [Bracket [ ] vs Dot (.) Method]

- Column selection in Pandas means accessing a particular column or columns from a DataFrame for viewing, analysis, or further operations.
- There are two common ways to select a column :-

1. Bracket notation [ ]
2. Dot notation .

- Bracket notation is used to select a column by writing its column name inside square brackets. Syntax :- df["column_name"]
- Dot notation is used to select a column by writing the column name after the DataFrame name and a dot. Syntax :- df.column_name.

Example :-

```
import pandas as pd

dict1 = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant", "Parshav", "Mittal", "Archana"], "Marks": [98, 89, 99, 87, 90, 83, 82] }

Marks = pd.DataFrame(dict1)

print(Marks)

 # Select Marks column using bracket notation

print("Using Bracket :")

print(df["Name"])

 # Select Marks column using dot notation

print("\nUsing Dot :")

print(df.Marks)

 # Select multiple columns using bracket notation

print(df[["Name", "Marks"]])
```

Output :-

```
     Name  Marks
0  Priyang     98
1   Aadhya     89
2   Krisha     99
3   Vedant     87
4  Parshav     90
5   Mittal     83
6  Archana     82
```

column using bracket notation Output :-

```
0    Priyang
1    Aadhya
2    Krisha
3    Vedant
4    Parshav
5    Mittal
6    Archana
Name: Name, dtype: object
```

column using dot notation Output :-

```
0    98
1    89
2    99
3    87
4    90
5    83
6    82
Name: Marks, dtype: int64
```

columns using bracket notation Output :-

```
     Name  Marks
0  Priyang     98
1   Aadhya     89
2   Krisha     99
3   Vedant     87
4  Parshav     90
5   Mittal     83
6  Archana     82
```

loc Vs. iloc Pandas Data Frame

- loc and iloc are Pandas DataFrame indexers used to select rows and columns from a DataFrame.
- loc is used for label-based selection.
- iloc is used for position-based selection.

Example :-

```
import pandas as pd

data = {
   "Name": ["Priyang", "Aadhya", "Krisha", "Vedant", "Parshav"],
   "Marks": [98, 89, 99, 87, 90],
   "Age": [20, 19, 21, 20, 22]
}

df = pd.DataFrame( data, index=["stu1", "stu2", "stu3", "stu4", "stu5"] )

print(df)
```

Output :-

	Name	Marks	Age
stu1	Priyang	98	20
stu2	Aadhya	89	19
stu3	Krisha	99	21
stu4	Vedant	87	20
stu5	Parshav	90	22

1. loc

- The loc indexer is used to select rows and columns using their labels or names. Syntax :- df.loc[row_label, column_label]

Example 1 :- Select One Row

```
print(df.loc["stu2"])
```

Output :-

```
Name     Aadhya
Marks        89
Age          19
```

Example 2 :- Select One Value

```
print(df.loc["stu3", "Marks"])   // Output :- 99
```

Example 3 :- Select Multiple Rows

```
print(df.loc[["stu1", "stu3"]])
```

Output :-

	Name	Marks	Age
stu1	Priyang	98	20
stu3	Krisha	99	21

Example 4 :- Select Specific Rows and Columns

print(df.loc[["stu1", "stu3"], ["Name", "Marks"]])

Output :-

	Name	Marks
stu1	Priyang	98
stu3	Krisha	99

2. iloc

- The iloc indexer is used to select rows and columns based on their integer positions. Syntax :- df.iloc[row_position, column_position]

Example 1 :- Select One Row

print(df.iloc[1])

Output :-

Name Aadhya

Marks 89

Age 19

Example 2 :- Select One Value

print(df.iloc[2, 1]) // **Output :-** 99

Example 3 :- Select Multiple Rows

print(df.iloc[[0, 2]])

Output :-

	Name	Marks	Age
stu1	Priyang	98	20
stu3	Krisha	99	21

Example 4 :- Select Specific Rows and Columns

print(df.iloc[[0, 2], [0, 1]])

Output :-

	Name	Marks
stu1	Priyang	98
stu3	Krisha	99

Filtering Data / Conditional Selection :-

- Filtering data means selecting only those rows from a DataFrame that satisfy a specific condition.
- Pandas allows us to filter data using conditional expressions, both with and without loc and iloc.

Example :-

```
import pandas as pd

dict1 = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant", "Parshav", "Mittal", "Archana"], "Marks": [98, 89, 99, 87, 90, 83, 82],

        "Grades": ["AA", "AB", "AA", "AB", "AC", "BA", "BB"]  }

df1 = pd.DataFrame( dict1, index=["stu1", "stu2", "stu3", "stu4", "stu5", "stu6", "stu7"] )

print(df1)
```

Output :-

```
        Name  Marks Grades
stu1  Priyang     98     AA
stu2   Aadhya     89     AB
stu3   Krisha     99     AA
stu4   Vedant     87     AB
stu5  Parshav     90     AC
stu6   Mittal     83     BA
stu7  Archana     82     BB
```

1. Students having marks greater than 90.

```
print("1. Marks greater than 90 :")

print(df1[df1["Marks"] > 90])
```

Output :-

```
        Name  Marks Grades
stu1  Priyang     98     AA
stu3   Krisha     99     AA
```

2. Students having AB grade

```
print("\n2. Students having AB grade:")

print(df1[df1["Grades"] == "AB"])
```

Output :-

```
        Name  Marks Grades
stu2  Aadhya     89     AB
stu4  Vedant     87     AB
```

3. Students having marks greater than 80 AND "AB" grade

```
print("\n3. Marks > 80 and Grade AB:")

print(df1[(df1["Marks"] > 80) &  (df1["Grades"] == "AB")])
```

Output :-

```
        Name  Marks Grades
stu2  Aadhya     89     AB
stu4  Vedant     87     AB
```

4. Names of students having marks greater than 90.

```
print("\n4. Names with Marks > 90 :")

print(df1[df1["Marks"] > 90]["Name"])
```

Output :-

```
stu1    Priyang

stu3     Krisha

Name: Name, dtype: object
```

5. Names of students having "AB" grade.

```
print("\n5. Names with Grade AB :")

print(df1[df1["Grades"] == "AB"]["Name"])
```

Output :-

```
stu2    Aadhya

stu4    Vedant

Name: Name, dtype: object
```

6. Students having marks greater than 90 [loc]

```
print("\n6. loc - Marks > 90 :")

print(df1.loc[df1["Marks"] > 90])
```

Output :-

```
       Name  Marks Grades

stu1  Priyang     98     AA

stu3   Krisha     99     AA
```

7. Students having marks greater than 80 AND "AB" grade [loc]

```
print("\n7. loc - Marks > 80 and Grade AB:")

print(df1.loc[ (df1["Marks"] > 80) & (df1["Grades"] == "AB") ])
```

Output :-

```
       Name  Marks Grades

stu2  Aadhya     89     AB

stu4  Vedant     87     AB
```

8. Students having marks greater than 80 AND "AB" grade [iloc]

```
condition = ( (df1["Marks"] > 80) & (df1["Grades"] == "AB") )

print(df1.iloc[condition.values])
```

Output :-

```
       Name  Marks Grades

stu2  Aadhya     89     AB

stu4  Vedant     87     AB
```

9. Name of students having "AB" grade [loc]

```
print(df1.loc[df1["Grades"] == "AB", "Name"])
```

Output :-

```
stu2    Aadhya

stu4    Vedant

Name: Name, dtype: object
```

Add / Remove Columns from Pandas DataFrame :-

- Adding a column means creating a new column in an existing DataFrame, while removing a column means deleting an existing column from the DataFrame.

Example :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, 89, 99, 87], "Grade": ["AA", "AB", "AA", "AB"] }

df = pd.DataFrame(data)

print(df)

# Add Column

df["Age"] = [20, 19, 21, 20]

print(df)

# Drop Column

df = df.drop("Age", axis=1)

print(df)

# Remove Multiple Column

df = df.drop(["Marks", "Grade"], axis=1)

print(df)
```

Output :-

	Name	Marks	Grade
0	Priyang	98	AA
1	Aadhya	89	AB
2	Krisha	99	AA
3	Vedant	87	AB

Output :-

	Name	Marks	Grade	Age
0	Priyang	98	AA	20
1	Aadhya	89	AB	19
2	Krisha	99	AA	21
3	Vedant	87	AB	20

Output :-

	Name	Marks	Grade
0	Priyang	98	AA
1	Aadhya	89	AB
2	Krisha	99	AA
3	Vedant	87	AB

Using inplace=True

- By default, drop() returns a new DataFrame and does not modify the original DataFrame.
- The inplace=True parameter directly modifies the existing DataFrame.
- Example :- df.drop("Grade", axis=1, inplace=True)

Output :-

	Name	Marks	Age
0	Priyang	98	20
1	Aadhya	89	19
2	Krisha	99	21
3	Vedant	87	20

Add a Column Using a Series

```
age = pd.Series([20, 19, 21, 20])

df["Age"] = age

print(df)
```

Add Column Using insert()

- The insert() method is used to add a new column at a specific position in the DataFrame.
- Syntax :- df.insert(position, "Column_Name", values)

Example :- df.insert(1, "Age", [20, 19, 21, 20])

print(df)

Adding Column from Existing Columns to DataFrame

```
import pandas as pd

data = { "Name": ["Amit", "Rahul", "Neha"], "Python": [80, 90, 85], "Java": [75, 85, 80], "SQL": [90, 80, 88] }

df = pd.DataFrame(data)

# Add Total column using existing columns

df["Total"] = df["Python"] + df["Java"] + df["SQL"]

print(df)
```

Output :-

	Name	Python	Java	SQL	Total
0	Amit	80	75	90	245
1	Rahul	90	85	80	255
2	Neha	85	80	88	253

Using assign( ) Method

- The assign( ) method is used to add a new column to a DataFrame without directly modifying the original DataFrame.
- Syntax :- new_dataframe = dataframe.assign(column1=series1, column2=series2)

Example :-

```
import pandas as pd

data = { "Name": ["Amit", "Rahul", "Neha"], "Marks": [80, 90, 85] }

df = pd.DataFrame(data)

# Add Result column using assign()

s1 = pd.Series(["AA", "AB", "AA", "AB", "AC", "BA", "BB"])

s2 = pd.Series([11, 22, 33, 44, 55, 66, 77])

df2 = df1.assign( Grades=s1, temp=s2 )

print(df2)
```

Output :-

	Name	Marks	Grades	temp
stu1	Priyang	98	AA	11
stu2	Aadhya	89	AB	22
stu3	Krisha	99	AA	33
stu4	Vedant	87	AB	44
stu5	Parshav	90	AC	55
stu6	Mittal	83	BA	66
stu7	Archana	82	BB	77

Add / Remove Row from Pandas DataFrame :-

```
import pandas as pd

dict1 = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant", "Parshav", "Mittal", "Archana"], "Marks": [98, 89, 99, 87, 90, 83, 82] }

df1 = pd.DataFrame(dict1)

print(df1)
```

Output :-

	Name	Marks
0	Priyang	98
1	Aadhya	89
2	Krisha	99
3	Vedant	87
4	Parshav	90
5	Mittal	83
6	Archana	82

Add Rows Using concat()

The concat() function is used to combine two or more DataFrames vertically for adding rows.

```
new_row = pd.DataFrame({ "Name": ["Rahul"], "Marks": [91] })

df2 = pd.concat([df1, new_row], ignore_index=True)

print(df2)
```

Output :-

	Name	Marks
0	Priyang	98
1	Aadhya	89
2	Krisha	99
3	Vedant	87
4	Parshav	90
5	Mittal	83
6	Archana	82
7	Rahul	91

Add Multiple Rows Using concat()

```
new_rows = pd.DataFrame({ "Name": ["Rahul", "Kiran"], "Marks": [91, 86] })

df2 = pd.concat([df1, new_rows], ignore_index=True)

print(df2)
```

Add Row Using append()

- The append() method was traditionally used to add rows to a DataFrame, but it has been removed from modern Pandas versions.
- Ex :- df2 = df1.append({"Name": "Rahul", "Marks": 91}, ignore_index=True)

Remove a Row Using drop()

```
df2 = df1.drop(2)

print(df2)
```

Output :-

	Name	Marks
0	Priyang	98
2	Krisha	99
4	Parshav	90
5	Mittal	83
6	Archana	82

Remove Multiple Rows Using drop()

```
df2 = df1.drop([1, 3], inplace=True)

print(df2)
```

Remove Rows Based on Condition

```
df1 = df1.drop(df1[df1["Marks"] < 90].index)

print(df1)
```

Output :-

	Name	Marks
0	Priyang	98
2	Krisha	99
4	Parshav	90

Create a CSV File From Pandas Data Frame :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, 89, 99, 87], "Grade": ["AA", "AB", "AA", "AB"] }

df = pd.DataFrame(data)

print(df)
```

Output :-

	Name	Marks	Grade
0	Priyang	98	AA
1	Aadhya	89	AB
2	Krisha	99	AA
3	Vedant	87	AB

Create CSV File

- The to_csv() method is used to save the DataFrame into a CSV file. Ex :- df.to_csv("students.csv")
- Create CSV Without Index :- df.to_csv("students.csv", index=False)

Read the CSV file

```
data = pd.read_csv("students.csv")

print(data)
```

Note :- pwd is used to display the current working directory, which tells the folder/location where Jupyter Notebook is currently working.

Read CSV, Excel, and TSV Files in Jupyter Notebook from Any Directory Using Pandas :-

1. Reading a CSV File

- A CSV (Comma-Separated Values) file stores data where values are separated by commas.

Example :-

```
import pandas as pd

# Read CSV file from a specific directory

df = pd.read_csv( 'C:\Users\zeel\Documents\Data\students.csv' )

print(df)
```

2. Reading an Excel File

- An Excel file usually has .xlsx or .xls extension.

Example :-

```
import pandas as pd

df = pd.read_excel( 'C:\Users\YourName\Documents\Data\students.xlsx' )

print(df)
```

For a specific sheet :-

```
df = pd.read_excel( r"C:\Users\YourName\Documents\Data\students.xlsx", sheet_name="Sheet1" )

print(df)
```

3. Reading a TSV File

- A TSV (Tab-Separated Values) file stores data where values are separated by tab characters.

Example :-

```
import pandas as pd

df = pd.read_csv( r"C:\Users\YourName\Documents\Data\students.tsv", sep="\t" )

print(df)
```

Here :- sep="\t" tells Pandas that the values are separated by a tab.

---

nrows Parameter in read_csv() :-

- The nrows parameter in pd.read_csv() is used to specify the number of rows that should be read from a CSV file.
- It is useful when a CSV file contains a large amount of data and we want to read only the first few rows.
- The nrows parameter allows us to read only a limited number of rows from a CSV file.
- Syntax :- pd.read_csv("filename.csv", nrows=number)

Example :-

```
import pandas as pd

df = pd.read_csv("students.csv", nrows=3)

print(df)
```

Output :-

```
     Name  Marks Grade

0  Priyang     98    AA

1  Aadhya     89    AB

2   Krisha     99    AA
```

---

skiprows Parameter in Pandas read_csv()

- The skiprows parameter in pd.read_csv() is used to skip one or more rows while reading a CSV file.
- It is useful when a CSV file contains unwanted rows at the beginning or at specific positions.
- Syntax :- pd.read_csv("filename.csv", skiprows=number)

Example 1 :- Skip First 2 Rows

```
import pandas as pd

df = pd.read_csv("students.csv", skiprows=2)

print(df)
```

Example 2 :- Skip Specific Rows

```
import pandas as pd

df = pd.read_csv( "students.csv", skiprows=[1, 3] )

print(df)
```

For Skip First 2 Rows Suppose students.csv contains :-

```
This is student data

Generated by School

Name,Marks,Grade

Priyang,98,AA

Aadhya,89,AB

Krisha,99,AA

Vedant,87,AB
```

Output :-

```
     Name  Marks Grade

0  Priyang     98    AA

1  Aadhya     89    AB

2   Krisha     99    AA

3   Vedant     87    AB
```

Usecols Parameter in Pandas read_csv( )

- The usecols parameter in pd.read_csv() is used to read only selected columns from a CSV file instead of loading all columns.
- It is useful when we need only specific columns from a large dataset.
- Syntax :- pd.read_csv("filename.csv", usecols=["column1", "column2"])

Example :-

Suppose students.csv contains :-

Name,Marks,Grade,Age

Priyang,98,AA,20

Aadhya,89,AB,19

Krisha,99,AA,21

Vedant,87,AB,20

Read only Name and Marks

```
import pandas as pd

df = pd.read_csv( "students.csv",    usecols=["Name", "Marks"] )

print(df)
```

Output :-

	Name	Marks
0	Priyang	98
1	Aadhya	89
2	Krisha	99
3	Vedant	87

---

index_col Parameter

- The index_col parameter of pd.read_csv() is used to specify which column should be used as the index of the DataFrame while reading the CSV file.

Example :-

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)

df = pd.read_csv( "students.csv", index_col="RollNo" )

print(df)
```

Suppose students.csv contains :-

RollNo,Name,Marks,Grade

101,Priyang,98,AA

102,Aadhya,89,AB

103,Krisha,99,AA

104,Vedant,87,AB

Original Output :-

	RollNo	Name	Marks	Grade
0	101	Priyang	98	AA
1	102	Aadhya	89	AB
2	103	Krisha	99	AA
3	104	Vedant	87	AB

Index_col Output :-

RollNo	Name	Marks	Grade
101	Priyang	98	AA
102	Aadhya	89	AB
103	Krisha	99	AA
104	Vedant	87	AB

set_index( ) Method :-

- The set_index() method is used to change an existing DataFrame column into the index after the DataFrame has already been created.

Example :-

```
import pandas as pd

data = { "RollNo": [101, 102, 103, 104], "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, 89, 99, 87] }

df = pd.DataFrame(data)

print("Before set_index() :")

print(df)

df = df.set_index("RollNo")

print("\nAfter set_index() :")

print(df)
```

Output :-

```
Before set_index() :
   RollNo    Name  Marks
0     101  Priyang     98
1     102   Aadhya     89
2     103   Krisha     99
3     104   Vedant     87
```

Output :-

```
After set_index() :-
          Name  Marks
RollNo
101     Priyang     98
102      Aadhya     89
103      Krisha     99
104      Vedant     87
```

---

names parameter :-

- The names parameter in pd.read_csv() is used to provide custom column names to a CSV file while reading it.
- It is useful when :-

1. The CSV file does not have column names.
2. We want to replace the existing column names with our own names.

- Syntax :- pd.read_csv("filename.csv", names=["Column1", "Column2", "Column3"])

Using names When CSV Has No Header :-

Suppose students.csv contains :-

```
Priyang,98,AA
Aadhya,89,AB
Krisha,99,AA
Vedant,87,AB
```

There are no column names in the file.

```
import pandas as pd

df = pd.read_csv( "students.csv",    names=["Name", "Marks", "Grade"])

print(df)
```

Output :-

```
     Name  Marks Grade
0  Priyang     98     AA
1   Aadhya     89     AB
2   Krisha     99     AA
3   Vedant     87     AB
```

Replacing Existing Column Names :-

Suppose the CSV contains :-

Name,Marks,Grade

Priyang,98,AA

Aadhya,89,AB

Krisha,99,AA

Output :-

	Student_Name	Score	Result
0	Priyang	98	AA
1	Aadhya	89	AB
2	Krisha	99	AA

We can replace the existing column names using :-

```
import pandas as pd

df = pd.read_csv( "students.csv", header=0, names=["Student_Name", "Score", "Result"])

print(df)
```

---

dtype Parameter :-

- The dtype parameter in pd.read_csv() is used to specify the data type of one or more columns while reading a CSV file.
- It allows us to control whether a column should be read as int, float, str, etc.
- Syntax :- pd.read_csv("filename.csv", dtype={"column_name": "data_type"})

Example :-

```
import pandas as pd

df = pd.read_csv( "students.csv", dtype={ "Name": "string",  "Marks": "int64", "RollNo": "int64" } )

print(df)

print(df.dtypes)
```

Output :-

	Name	Marks	RollNo
0	Priyang	98	101
1	Aadhya	89	102
2	Krisha	99	103
3	Vedant	87	104

Data Types :-

Name	string
Marks	int64
RollNo	int64

dtype: object

chunksize Parameter :-

- The chunksize parameter in pd.read_csv() is used to read a large CSV file in smaller portions, called chunks, instead of loading the entire file into memory at once.
- This is useful when the CSV file is very large and cannot be efficiently loaded into memory at one time.
- Syntax :- pd.read_csv("filename.csv", chunksize=number)

Example :-

1. Read CSV in Chunks

```
import pandas as pd

# Read CSV file in chunks of 3 rows

chunks = pd.read_csv("students.csv", chunksize=3)

# Display each chunk

for chunk in chunks:

    print(chunk)

    print()
```

Suppose students.csv contains :-

```
Name,Marks

Priyang,98

Aadhya,89

Krisha,99

Vedant,87

Parshav,90

Mittal,83

Archana,82

Rahul,91

Kiran,86
```

Output :-

Chunk 1 :-

```
     Name  Marks

0  Priyang    98

1   Aadhya    89

2   Krisha    99
```

Chunk 2 :-

```
     Name  Marks

3   Vedant    87

4  Parshav    90

5   Mittal    83
```

Chunk 3 :-

```
     Name  Marks

6  Archana    82

7    Rahul    91

8    Kiran    86
```

2. Process Each Chunk Separately

```
import pandas as pd

chunks = pd.read_csv("students.csv", chunksize=3)

for chunk in chunks:

    result = chunk[chunk["Marks"] > 90]

    print(result)
```

3. Concatenate Chunks Back Together :- After processing the chunks, we can combine them using pd.concat().

Example :-

```
import pandas as pd

chunks = pd.read_csv("students.csv", chunksize=3)

# Store each chunk in a list

chunk_list = [ ]

for chunk in chunks:

    chunk_list.append(chunk)

df = pd.concat(chunk_list, ignore_index=True)
```

Output :-

```
     Name  Marks

0  Priyang    98

1   Aadhya    89

2   Krisha    99

3   Vedant    87

4  Parshav    90

5   Mittal    83

6  Archana    82

7    Rahul    91

8    Kiran    86
```

inplace Parameter :-

- The inplace parameter in Pandas is used to specify whether an operation should modify the original DataFrame or return a new modified DataFrame.

Example :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, 89, 99, 87] }

df = pd.DataFrame(data)

# Remove row 1 without changing the original DataFrame

new_df = df.drop(1, inplace=True)

print("Original DataFrame :-")

print(df)

print("\nNew DataFrame :-")

print(new_df)
```

Output :-

Original DataFrame :-

	Name	Marks
0	Priyang	98
1	Aadhya	89
2	Krisha	99
3	Vedant	87

New DataFrame :-

	Name	Marks
0	Priyang	98
2	Krisha	99
3	Vedant	87

---

isnull() & notnull() in Pandas :-

- Missing values are empty or unavailable values in a dataset.
- Pandas provides isnull() and notnull() to identify missing and non-missing values in a DataFrame or Series.

Example :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, None, 99, None], "Grade": ["AA", "AB", None, "AB"] }

df = pd.DataFrame(data)

print(df)
```

Output :-

	Name	Marks	Grade
0	Priyang	98.0	AA
1	Aadhya	NaN	AB
2	Krisha	99.0	None
3	Vedant	NaN	AB

1. isnull()

- The isnull() method is used to identify missing values in a Pandas DataFrame or Series. It returns True for missing values and False for non-missing values.

Example :-

```
print(df.isnull())
```

Output :-

	Name	Marks	Grade
0	False	False	False
1	False	True	False
2	False	False	True
3	False	True	False

2. notnull()

- The notnull() method is used to identify non-missing values. It returns True for available values and False for missing values.

Example :-

print(df.notnull())

Output :-

	Name	Marks	Grade
0	True	True	True
1	True	False	True
2	True	True	False
3	True	False	True

3. Count Missing Values

- We can use sum() with isnull() to count missing values.

Example :- print(df.isnull().sum())

Output :-

Name	0
Marks	2
Grade	1

dtype: int64

4. Find Rows Having Missing Values

print(df[df.isnull().any(axis=1)])

Output :-

	Name	Marks	Grade
1	Aadhya	NaN	AB
2	Krisha	99.0	None
3	Vedant	NaN	AB

5. Find Rows Where Marks Are Missing

print(df[df["Marks"].isnull()])

Output :-

	Name	Marks	Grade
1	Aadhya	NaN	AB
3	Vedant	NaN	AB

na_values :-

- The na_values parameter in pd.read_csv() is used to specify additional values that should be treated as missing values (NaN) while reading a CSV file.
- It is useful when a dataset uses values such as "NA", "N/A", "missing", "-", or "Not Available" to represent missing data.

Example :-

```
import pandas as pd

df = pd.read_csv( "students.csv", na_values=["NA", "N/A", "-"] )

print(df)
```

Suppose students.csv contains :-

```
Name,Marks,Grade

Priyang,98,AA

Aadhya,NA,AB

Krisha,99,N/A

Vedant,-,AB
```

Output :-

```
     Name  Marks Grade
0  Priyang   98.0    AA
1   Aadhya    NaN    AB
2   Krisha   99.0   NaN
3   Vedant    NaN    AB
```

---

na_filter Parameter :-

- The na_filter parameter determines whether Pandas should detect missing-value markers while reading a CSV file.

Example :-

```
import pandas as pd

df = pd.read_csv( "students.csv", na_filter=True )

print(df)
```

For example, if the CSV contains :-

```
Name,Marks

Priyang,98

Aadhya,

Krisha,99
```

Output :-

```
     Name  Marks
0  Priyang   98.0
1   Aadhya    NaN
2   Krisha   99.0
```

keep_default_na Parameter :-

- The keep_default_na parameter determines whether Pandas should continue to recognize its default missing-value strings as NaN.

Example :-

```
import pandas as pd

df = pd.read_csv( "students.csv", keep_default_na=True )

print(df)
```

Suppose students.csv contains :-

```
Name,Marks

Priyang,98

Aadhya,NA

Krisha,99
```

Output :-

```
     Name  Marks
0  Priyang   98.0
1   Aadhya    NaN
2   Krisha   99.0
```

na_values with keep_default_na :-

```
df = pd.read_csv( "students.csv", na_values=["Missing"], keep_default_na=True )

print(df)
```

Suppose :-

```
Name,Marks

Priyang,98

Aadhya,NA

Krisha,Missing
```

Output :-

```
    Name  Marks

0  Priyang   98.0

1   Aadhya    NaN

2   Krisha    NaN
```

keep_default_na=False with na_values :-

```
df = pd.read_csv( "students.csv", na_values=["Missing"], keep_default_na=False )

print(df)
```

Output :-

```
    Name    Marks

0  Priyang     98

1   Aadhya     NA

2   Krisha    NaN
```

Now :-

NA → remains "NA"

Missing → becomes NaN

---

dropna() – Handling Missing Values Using Pandas :-

- The dropna() method in Pandas is used to remove rows or columns that contain missing values such as NaN or None.

Example :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, None, 99, None], "Grade": ["AA", "AB", None, "AB"] }

df = pd.DataFrame(data)

print(df)
```

Output :-

```
     Name  Marks Grade

0  Priyang   98.0    AA

1   Aadhya    NaN    AB

2   Krisha   99.0  None

3   Vedant    NaN    AB
```

dropna() – Remove Rows

```
df2 = df.dropna()

print(df2)
```

Output :-

```
     Name  Marks Grade

0  Priyang   98.0    AA
```

dropna(axis=1) – Remove Columns

```
df2 = df.dropna(axis=1)

print(df2)
```

Output :-

```
     Name

0  Priyang

1   Aadhya

2   Krisha

3   Vedant
```

how="all" - how="all" removes a row only when all values in that row are missing.

```
df.dropna(how="all")
```

Output :-

```
Name     Marks    Grade

Priyang  98       AA

Aadhya   NaN      AB

NaN      NaN      NaN ← Removed
```

Subset :- The subset parameter is used to check missing values only in specified columns.

Suppose we want to remove rows where Marks is missing :-

```
df2 = df.dropna(subset=["Marks"])

print(df2)
```

Output :-

```
    Name  Marks Grade
0  Priyang   98.0    AA
2   Krisha   99.0  None
```

inplace=True :-

```
df.dropna(inplace=True)

print(df)
```

Output :-

```
    Name  Marks Grade
0  Priyang   98.0    AA
```

thresh Parameter :- The thresh parameter specifies the minimum number of non-missing values required to keep a row or column.

```
df2 = df.dropna(thresh=2)

print(df)
```

Output :-

```
Name     Marks    Grade
Priyang  98       AA     → 3 non-missing → Keep
Aadhya   NaN      AB     → 2 non-missing → Keep
Krisha   99       NaN    → 2 non-missing → Keep
NaN      NaN      NaN    → 0 non-missing → Remove
```

1. Drop the columns where all values are missing.

df.dropna(axis=1, how="all") // axis=1 → checks columns. , how="all" → removes the column only when all values are missing.

2. Drop the rows where all values are missing.

df.dropna(axis=0, how="all")

3. Keep only the rows with at least 2 non-NA values.

df.dropna(axis=0, thresh=2)

---

fillna() – Handling Missing Values in Pandas :-

- The fillna() function in Pandas is used to replace missing values such as NaN or None with a specified value or another calculated value.
- It is useful when we do not want to delete rows or columns containing missing values.

Example :-

```
import pandas as pd

data = { "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, None, 99, None], "Grade": ["AA", "AB", None, "AB"] }

df = pd.DataFrame(data)

print(df)
```

Output :-

```
    Name  Marks Grade
0  Priyang   98.0    AA
1  Aadhya    NaN    AB
2  Krisha   99.0  None
3  Vedant    NaN    AB
```

1. Fill All Missing Values with a Single Value

```
df2 = df.fillna(0)

print(df2)
```

Output 1 :-

	Name	Marks	Grade
0	Priyang	98.0	AA
1	Aadhya	0.0	AB
2	Krisha	99.0	0
3	Vedant	0.0	AB

Output 2 :-

	Name	Marks	Grade
0	Priyang	98.0	AA
1	Aadhya	0.0	AB
2	Krisha	99.0	None
3	Vedant	0.0	AB

2. Fill Missing Values in a Specific Column

```
df["Marks"] = df["Marks"].fillna(0)

print(df)
```

3. Fill Missing Values with a String

```
df["Grade"] = df["Grade"].fillna("Not Available")

print(df)
```

4. Fill Different Columns with Different Values

```
df = pd.DataFrame({"Name": ["Priyang", "Aadhya", "Krisha"], "Marks": [98, None, 99], "Grade": ["AA", None, "AA"] })

df = df.fillna({ "Marks": 0, "Grade": "Not Available"})

print(df)
```

Output :-

	Name	Marks	Grade
0	Priyang	98.0	AA
1	Aadhya	0.0	Not Available
2	Krisha	99.0	AA

5. Fill Missing Values with Mean

```
 import pandas as pd

df = pd.DataFrame({ "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, None, 99, 87] })

df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

print(df)
```

For example, if :-

Marks

98

NaN

99

87

Output :-

	Name	Marks
0	Priyang	98.000000
1	Aadhya	94.666667
2	Krisha	99.000000
3	Vedant	87.000000

The mean is :- (98 + 99 + 87) / 3 = 94.67

6. Fill Missing Values with Median

```
df["Marks"] = df["Marks"].fillna(df["Marks"].median())
```

7. Fill Missing Values with Mode

```
df["Grade"] = df["Grade"].fillna(df["Grade"].mode()[0])
```

8. Forward fill (ffill) :- It replaces a missing value with the previous available value.

df["Marks"] = df["Marks"].ffill()

Example :-	After forward fill :-
98	98
NaN	98
99	99
NaN	99
87	87

9. Backward Fill :- Backward fill (bfill) replaces a missing value with the next available value.

df["Marks"] = df["Marks"].bfill()

Example :-	After backward fill :-
98	98
NaN	99
99	99
NaN	87
87	87

Pandas Replace :-

- The replace() method in Pandas is used to replace specific values in a DataFrame or Series with other values.
- It can be used to replace missing-value representations such as NA, N/A, -, Unknown, or None, as well as normal values.
- Syntax :- df.replace(old_value, new_value)

Basic replace() Example :-

```
import pandas as pd

df = pd.DataFrame({ "Name": ["Priyang", "Aadhya", "Krisha", "Vedant"], "Marks": [98, "NA", 99, "NA"], "Grade": ["AA", "AB", "NA", "AB"] })

print(df)
```

Output :-

```
     Name  Marks Grade
0  Priyang   98.0    AA
1   Aadhya    NaN    AB
2   Krisha   99.0   NaN
3   Vedant    NaN    AB
```

We can replace it with NaN :-

```
import numpy as np

df = df.replace("NA", np.nan)

print(df)
```

Output Basic Replace :-

```
     Name Marks Grade
0  Priyang    98    AA
1   Aadhya    NA    AB
2   Krisha    99    NA
3   Vedant    NA    AB
```

Replace Missing Values with a Specific Value :-

```
df = df.replace(np.nan, 40)

print(df)
```

Output :-

```
     Name  Marks Grade
0  Priyang   98.0    AA
1   Aadhya    40    AB
2   Krisha   99.0    40
3   Vedant    40    AB
```

Replace Different Values with Different Values :-

```
df = pd.DataFrame({ "Name": ["Priyang", "Aadhya", "Krisha"], "Marks": [98, NA, 99], "Grade": ["AA", "Unknown", "AB"] })

df = df.replace({ "Marks": {np.nan: 40}, "Grades": {np.nan: "FF"} })

print(df)
```

Replace Multiple Values with One Value :-

```
import numpy as np

df = df.replace( ["NA", "N/A", "Missing", "Unknown"], np.nan)

print(df)
```

Before Replace :-

	Name	Marks	Grade
0	Priyang	98	AA
1	Aadhya	NA	N/A
2	Krisha	99	Unknown
3	Vedant	Missing	AB

Output :-

	Name	Marks	Grade
0	Priyang	98.0	AA
1	Aadhya	NaN	NaN
2	Krisha	99.0	NaN
3	Vedant	NaN	AB

inplace=True :-

```
df.replace("NA", np.nan, inplace=True)
```

Replace the names "Priyang" and "Aadhya" with "Anil" :-

```
df.replace( ["Priyang", "Aadhya"], "Anil", inplace=True )
```

Replace NaN values of Marks column with 0 :-

```
df.replace({'Marks' : np.nan})
```

---

interpolate() – Handling Missing Values in Pandas :-

- The interpolate() method in Pandas is used to fill missing values by estimating values based on the existing values around them.
- It is especially useful for numerical and time-series data, where values usually follow a continuous pattern.

Basic interpolate() Example :-

```
import pandas as pd

data = { "Marks": [50, None, 70, None, 90] }

df = pd.DataFrame(data)

print(df)

df["Marks"] = df["Marks"].interpolate()

print(df)
```

Original Output :-

	Marks
0	50.0
1	NaN
2	70.0
3	NaN
4	90.0

Output :-

	Marks
0	50.0
1	60.0
2	70.0
3	80.0
4	90.0

How did Pandas calculate it?

For the first missing value :- 50 ⟶ NaN ⟶ 70

Pandas estimates the middle value :- (50 + 70) / 2 = 60

limit Parameter :- The limit parameter specifies the maximum number of consecutive missing values to fill.

Example :-

```
df = pd.DataFrame({ "Marks": [10, None, None, None, 50] })

df["Marks"] = df["Marks"].interpolate(limit=2)

print(df)
```

Output :-

	Marks
0	10.0
1	20.0
2	30.0
3	NaN
4	50.0

limit_direction :-

The limit_direction parameter specifies the direction in which missing values should be filled when using a limit.

Common options are :- "forward", "backward", "both"

Example :-

```
df = pd.DataFrame({ "Marks": [10, None, None, None, 50] })

df["Marks"] = df["Marks"].interpolate( limit=2, limit_direction="both" )

print(df)
```

GroupBy – GroupBy Without Using GroupBy :-

- The groupby() method in Pandas is used to divide data into groups based on one or more columns and then perform operations such as sum(), mean(), count(), max(), and min() on each group.

Example :-

```
import pandas as pd

df = pd.DataFrame({ "Department": ["Computer", "IT", "Computer", "IT", "Computer"], "Name": ["A", "B", "C", "D", "E"], "Marks": [80, 70, 90, 85, 75] })

print(df)
```

Output :-

	Department	Name	Marks
0	Computer	A	80
1	IT	B	70
2	Computer	C	90
3	IT	D	85
4	Computer	E	75

Using GroupBy :- Suppose we want to calculate the average marks of each department.

```
result = df.groupby("Department")["Marks"].mean()

print(result)
```

Output :-

```
Department

Computer    81.666667

IT          77.500000

Name: Marks, dtype: float64
```

GroupBy Without Using groupby() :-

We can achieve the same result using methods such as :-

- unique()
- Boolean filtering
- mean()

Example :- departments = df["Department"].unique()

for department in departments:

    marks = df[df["Department"] == department]["Marks"]

    print(department, marks.mean())

Output :-

Computer 81.66666666666667

IT 77.5

GroupBy Without groupby() Using a Dictionary :-

result = { }

for department in df["Department"].unique():

    marks = df[df["Department"] == department]["Marks"]

    result[department] = marks.mean()

print(result) // Output :- {'Computer': 81.66666666666667, 'IT': 77.5}

Multiple Aggregations :-

df.groupby("Department")["Marks"].agg(["sum", "mean", "min", "max", "count"])

Output :-

	sum	mean	min	max	count
Computer	245	81.666667	75	90	3
IT	155	77.500000	70	85	2

count() Without groupby() :-

Using groupby() :-

df.groupby("Department")["Marks"].count()

Without groupby() :-

for department in df["Department"].unique():

    marks = df[df["Department"] == department]["Marks"]

    print(department, marks.count())

Output :-

Computer 3

IT 2

groupby() with sum() :-

result = df.groupby("Department")["Marks"].sum()

print(result)

Output :-

Department

Computer 245

IT 155

Name: Marks, dtype: int64

merge() – Merge DataFrames

- The merge() function in Pandas is used to combine two or more DataFrames based on one or more common columns or keys.
- Syntax :- pd.merge(df1, df2, on="column_name")

Example :-

Let's create two DataFrames.

DataFrame 1 – Students

```
import pandas as pd

df1 = pd.DataFrame({"ID": [1, 2, 3, 4], "Name": ["A", "B", "C", "D"]})

print(df1)
```

Output :-

	ID	Name
0	1	A
1	2	B
2	3	C
3	4	D

DataFrame 2 – Marks

```
df2 = pd.DataFrame({"ID": [1, 2, 3, 5],    "Marks": [80, 90, 75, 85]})

print(df2)
```

Output :-

	ID	Marks
0	1	80
1	2	90
2	3	75
3	5	85

Inner Merge :-

```
result = pd.merge(df1, df2, on="ID")print(result)
```

Output :-

	ID	Name	Marks
0	1	A	80
1	2	B	90
2	3	C	75

Left Merge :-

- A left merge keeps all rows from the left DataFrame and adds matching data from the right DataFrame.

```
result = pd.merge(df1, df2, on="ID", how="left")

print(result)
```

Output :-

	ID	Name	Marks
0	1	A	80.0
1	2	B	90.0
2	3	C	75.0
3	4	D	NaN

- ID 4 exists in df1 but not in df2, so its Marks becomes NaN.

Right Merge :-

- A right merge keeps all rows from the right DataFrame.

```
result = pd.merge(df1, df2, on="ID", how="right")

print(result)
```

Output :-

	ID	Name	Marks
0	1	A	80
1	2	B	90
2	3	C	75
3	5	NaN	85

Merge Using Different Column Names :- Sometimes the common columns have different names.

First DataFrame

```
df1 = pd.DataFrame({"Student_ID": [1, 2, 3],"Name": ["A", "B", "C"]})
```

Second DataFrame

```
df2 = pd.DataFrame({"ID": [1, 2, 3], "Marks": [80, 90, 75]})
```

Merge DataFrame

```
result = pd.merge(df1, df2, left_on="Student_ID", right_on="ID")

print(result)
```

Output :-

	Student_ID	Name	ID	Marks
0	1	A	1	80
1	2	B	2	90
2	3	C	3	75

---

concat() – Concatenate Series and DataFrames :-

- The concat() function in Pandas is used to combine two or more Series or DataFrames along rows or columns.
- It is mainly used when we want to stack or join data together without matching a common key, unlike merge().
- Syntax :- pd.concat([df1, df2])

Example :- Concatenate Two DataFrames Row-wise

```
import pandas as pd

df1 = pd.DataFrame({ "Name": ["A", "B"], "Marks": [80, 90] })

df2 = pd.DataFrame({ "Name": ["C", "D"], "Marks": [75, 85] })

print(df1)

print(df2)

result = pd.concat([df1, df2])

print(result)
```

Df 1 Output :-

	Name	Marks
0	A	80
1	B	90

Df 2 Output :-

	Name	Marks
0	C	75
1	D	85

Output :-

	Name	Marks
0	A	80
1	B	90
0	C	75
1	D	85

Concatenate DataFrames Column-wise :-

```
df1 = pd.DataFrame({ "Name": ["A", "B", "C"] })

df2 = pd.DataFrame({ "Marks": [80, 90, 75] })

result = pd.concat( [df1, df2], axis=1 )
```

Output :-

	Name	Marks
0	A	80
1	B	90
2	C	75

Concatenate Series :-

```
s1 = pd.Series([10, 20, 30])

s2 = pd.Series([40, 50, 60])

result = pd.concat([s1, s2])

print(result)
```

Output :-

```
0    10
1    20
2    30
0    40
1    50
2    60
dtype: int64
```

Concatenate DataFrames with Different Columns :-

```
df1 = pd.DataFrame({ "Name": ["A", "B"], "Marks": [80, 90] })

df2 = pd.DataFrame({ "Name": ["C", "D"], "Grade": ["A", "B"] })

result = pd.concat( [df1, df2], ignore_index=True )

print(result)
```

Output :-

	Name	Marks	Grade
0	A	80.0	NaN
1	B	90.0	NaN
2	C	NaN	A
3	D	NaN	B

join="inner" :-

```
result = pd.concat( [df1, df2], join="inner", ignore_index=True )

print(result)
```

concat() – keys Parameter :-

- The keys parameter in pd.concat() is used to add labels to the DataFrames or Series being concatenated.

Example :-

```
import pandas as pd

df1 = pd.DataFrame({"Name": ["A", "B"], "Marks": [80, 90]})

df2 = pd.DataFrame({"Name": ["C", "D"], "Marks": [75, 85]})
```

Output :-

	Name	Marks
0	A	80
1	B	90
0	C	75
1	D	85

1. Without keys

```
result = pd.concat([df1, df2])print(result)
```

Here, it is difficult to identify which rows came from df1 and which came from df2.

2. Using keys

```
result = pd.concat([df1, df2], keys=["First", "Second"],sort=True)

print(result)
```

Output :-

		Name	Marks
First	0	A	80
	1	B	90
Second	0	C	75
	1	D	85

How to Fetch Random Samples From a Dataset :-

- The sample() method in Pandas is used to randomly select rows or columns from a DataFrame or Series.
- It is useful when we want to take a random sample of data for testing, analysis, or machine learning.

Example :-

```
import pandas as pd

df = pd.DataFrame({ "Name": ["A", "B", "C", "D", "E"], "Marks": [80, 90, 75, 85, 95] })

print(df)
```

To select one random row :- df.sample()

Output :-

	Name	Marks
2	C	75

Dataset :-

	Name	Marks
0	A	80
1	B	90
2	C	75
3	D	85
4	E	95

Fetch Multiple Random Rows :- Use the n parameter.

df.sample(n=2)

random_state :- If you want the same random sample every time, use random_state.

Example :- df.sample(n=2, random_state=42)

Possible output :-

	Name	Marks
4	E	95
1	B	90

Fetch a Percentage of Rows :- Instead of specifying the number of rows, we can use frac.

Example :- df.sample(frac=0.4)

If the DataFrame contains 5 rows :- 5 × 0.4 = 2 rows Then So approximately 40% of the rows are selected.

Another Example :- df.sample(frac=0.5, random_state=42)

Random Sampling of Columns :-

df.sample(n=1, axis=1) // This selects one random column.

df.sample(axis=1) // To randomly select columns.

Complete Example :-

```
import pandas as pd

df = pd.DataFrame({"Name": ["A", "B", "C", "D", "E", "F"], "Marks": [80, 90, 75, 85, 95, 70] })

random_data = df.sample( n=3, random_state=42 )

print(random_data)
```

Here :-

n=3

↓

3 random rows

|

random_state=42

↓

Same 3 rows every time

Matplotlib :-

What is Matplotlib :-

- Matplotlib is a popular Python library used for data visualization. It helps us represent data in the form of graphs, charts, and plots.

Common graphs in Matplotlib :-

Graph	Function	Use
Line chart	plt.plot()	Show trends over time
Bar chart	plt.bar()	Compare categories
Histogram	plt.hist()	Show data distribution
Scatter plot	plt.scatter()	Show relationship between two variables
Pie chart	plt.pie()	Show proportions

Line Chart Example :-

Import matplotlib.pyplot as plt

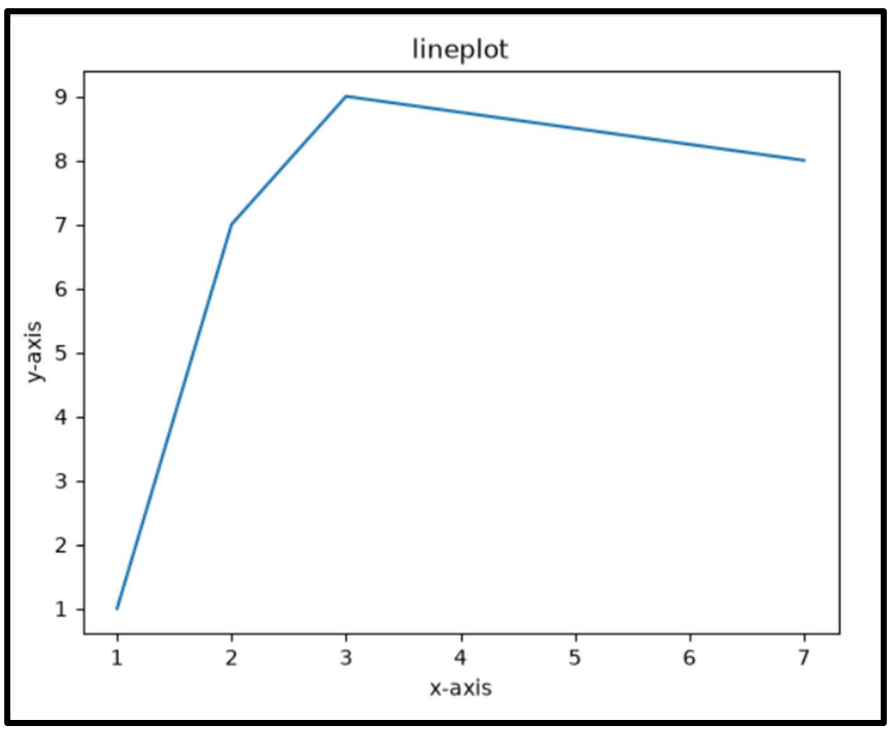
Import numpy as np

1. Draw A Line Plot :-

x = [1 , 2 , 3 , 7]

y = [1 , 7 , 9 , 8]

plt.plot(x , y)



2. Give Title , Xlabel and Ylabel :-

import matplotlib.pyplot as plt

x = [1, 2, 3, 7]

y = [1, 7, 9, 8]

plt.plot(x, y)

plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

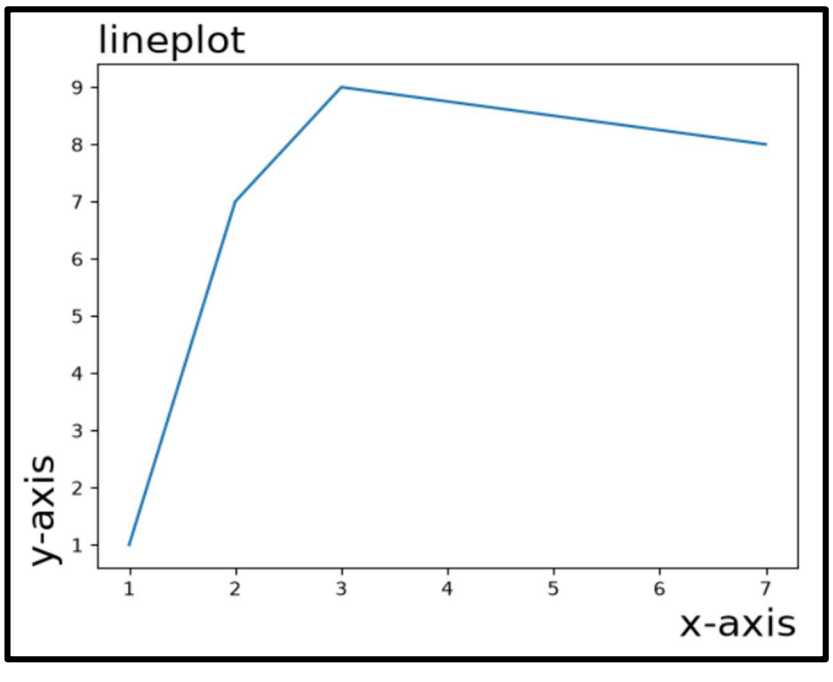
plt.show()

3. Change Font Size of Title, Xlabel and Ylabel and also change location.

plt.title("lineplot", fontsize=20, loc='left')

plt.xlabel("x-axis", fontsize=20, loc='right')

plt.ylabel("y-axis", fontsize=20, loc='bottom')



4. Change Color & Width of line :-

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 7]

y = [1, 7, 9, 8]

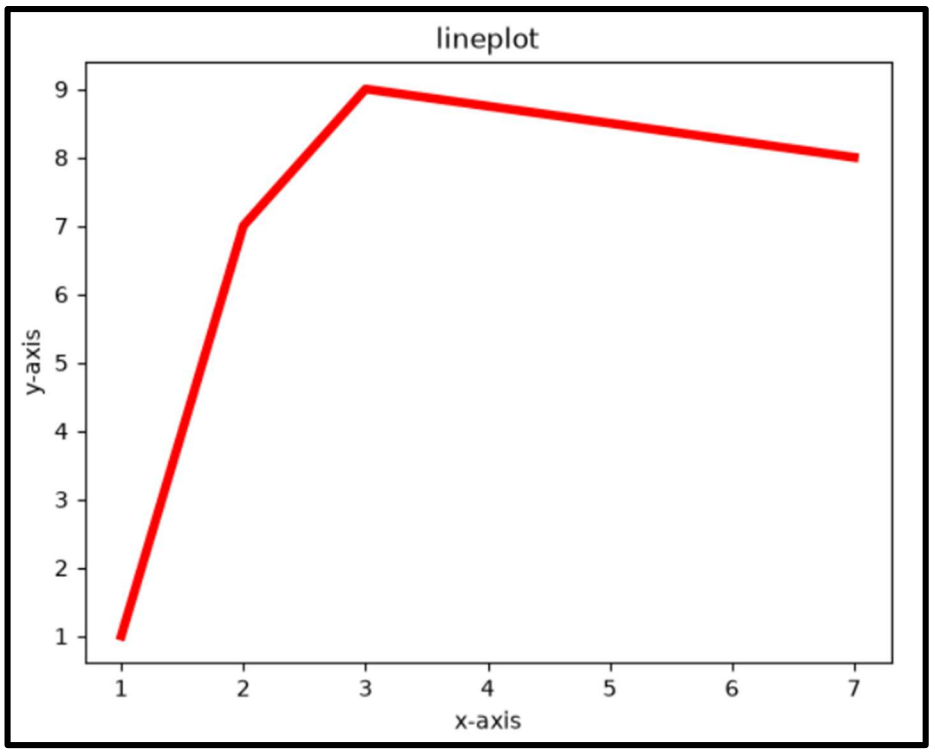
plt.plot(x, y, color='r', linewidth=4)

plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

plt.show()
```



5. Change Marker, Marker Size & Line Style :-

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 7]

y = [1, 7, 9, 8]

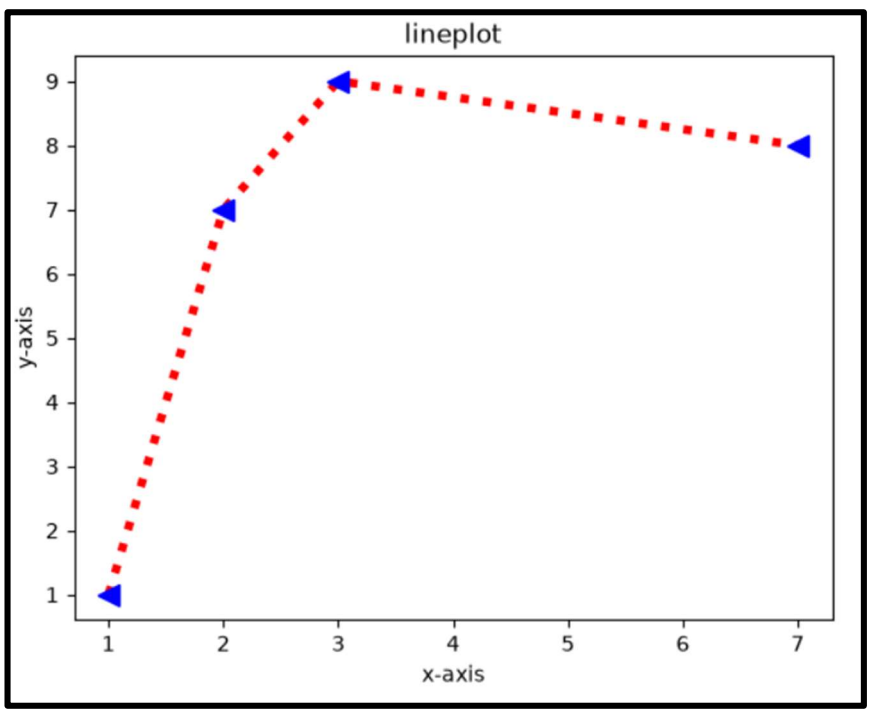
plt.plot(x, y, color='r', linewidth=4, marker='<',markersize=10,markerfacecolor='blue',markeredgecolor='blue',linestyle=':')

plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

plt.show()
```



6. Add Legend To Your Graph Also Change Location :-

```
import matplotlib.pyplot as plt

x = np.random.randint(low=1,high=10,size=20)

y = np.random.randint(low=1,high=8,size=20)

plt.plot(x, y, color='r', linewidth=4, marker='<',markersize=10,markerfacecolor='blue',markeredgecolor='blue',linestyle=':')

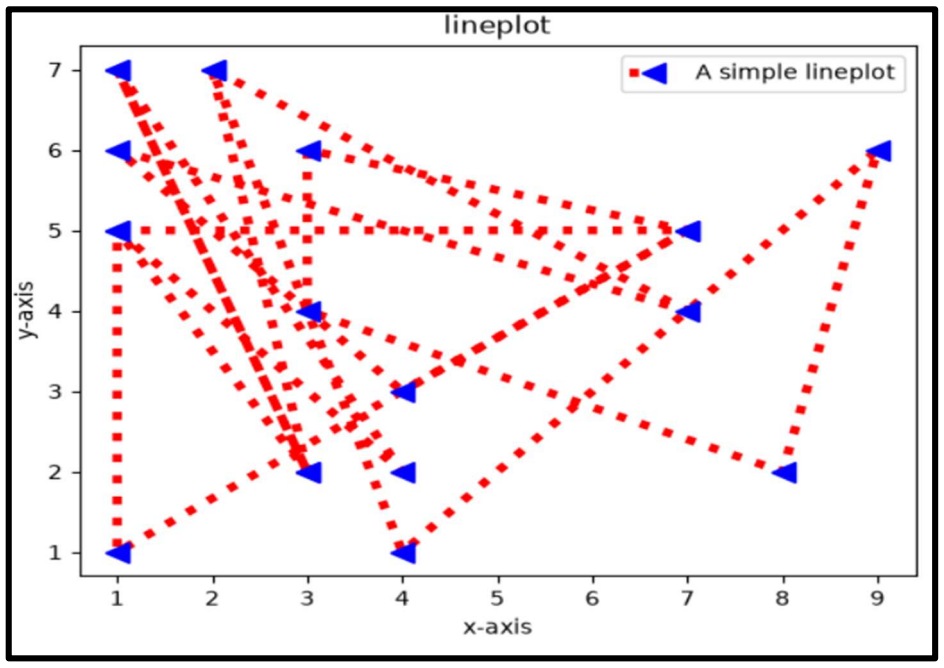
plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

plt.legend(["A simple lineplot"])

plt.show()
```



7. xlim And ylim :-

```
import matplotlib.pyplot as plt

import numpy as np

x = np.random.randint(low=1, high=10, size=20)

y = np.random.randint(low=1, high=8, size=20)

plt.plot(x, y, color='r', linewidth=4, marker='<', markersize=10, markerfacecolor='blue', markeredgecolor='blue', linestyle=' : ' )

plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

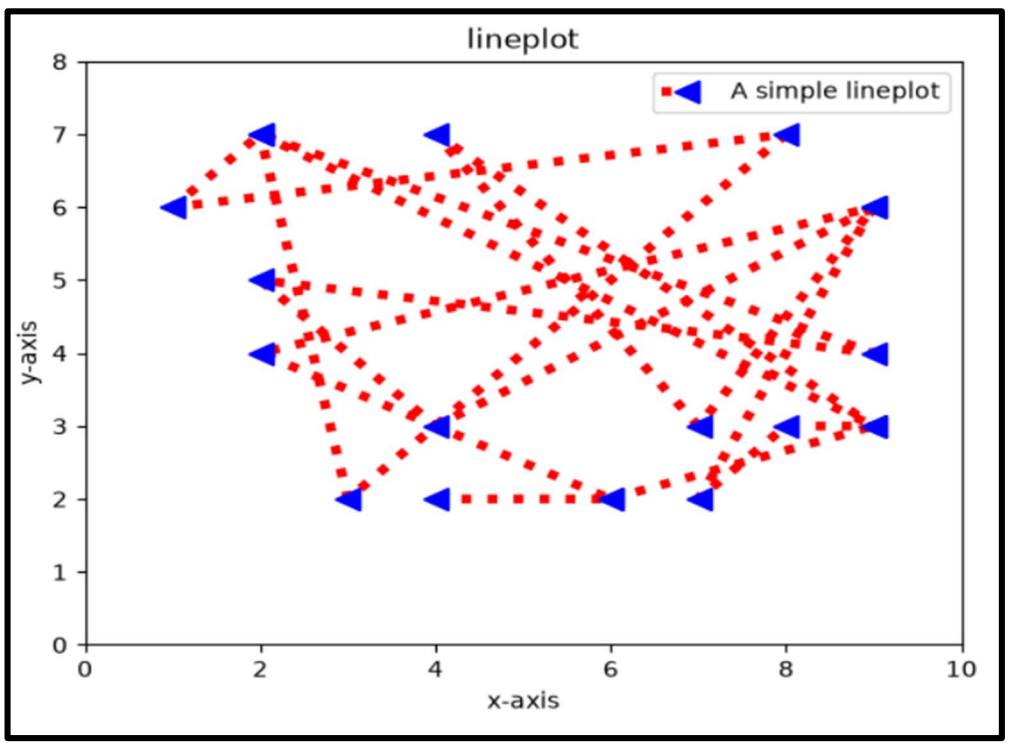
# Set axis limits

plt.xlim(0, 10)

plt.ylim(0, 8)

plt.legend(["A simple lineplot"])

plt.show()
```



8. Create separate lines for x and y and change plot style :-

```
import matplotlib.pyplot as plt

import numpy as np

x = np.random.randint(low=1, high=10, size=20)

y = np.random.randint(low=1, high=8, size=20)

plt.plot(x, color='r', linewidth=4, marker='<', markersize=10, markerfacecolor='red', markeredgecolor='blue', linestyle=':')

plt.plot(y, color='r', linewidth=4, marker='<', markersize=10, markerfacecolor='blue', markeredgecolor='blue', linestyle=':')

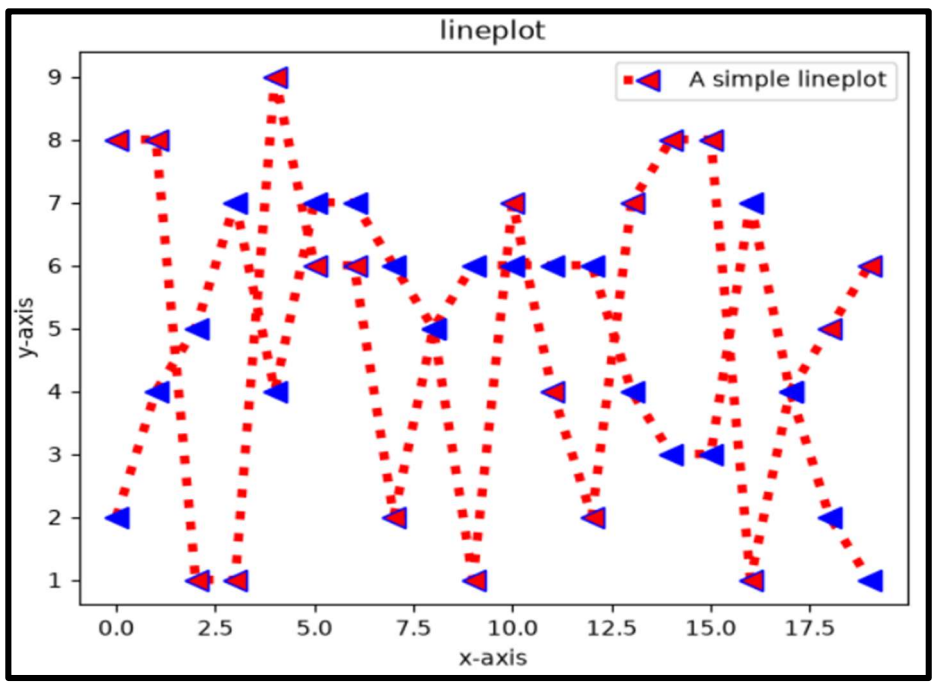
plt.title("lineplot")

plt.xlabel("x-axis")

plt.ylabel("y-axis")

plt.legend(["A simple lineplot"])

plt.show()
```



9. Change x and y limit using axis method :-

```
import matplotlib.pyplot as plt

import numpy as np

x = np.random.randint(low=1, high=10, size=20)

y = np.random.randint(low=1, high=8, size=20)

plt.plot(x, y, color='r', linewidth=4, marker='<', markersize=10, markerfacecolor='blue', markeredgecolor='blue', linestyle=':')

plt.title("lineplot")

plt.xlabel("x-axis")

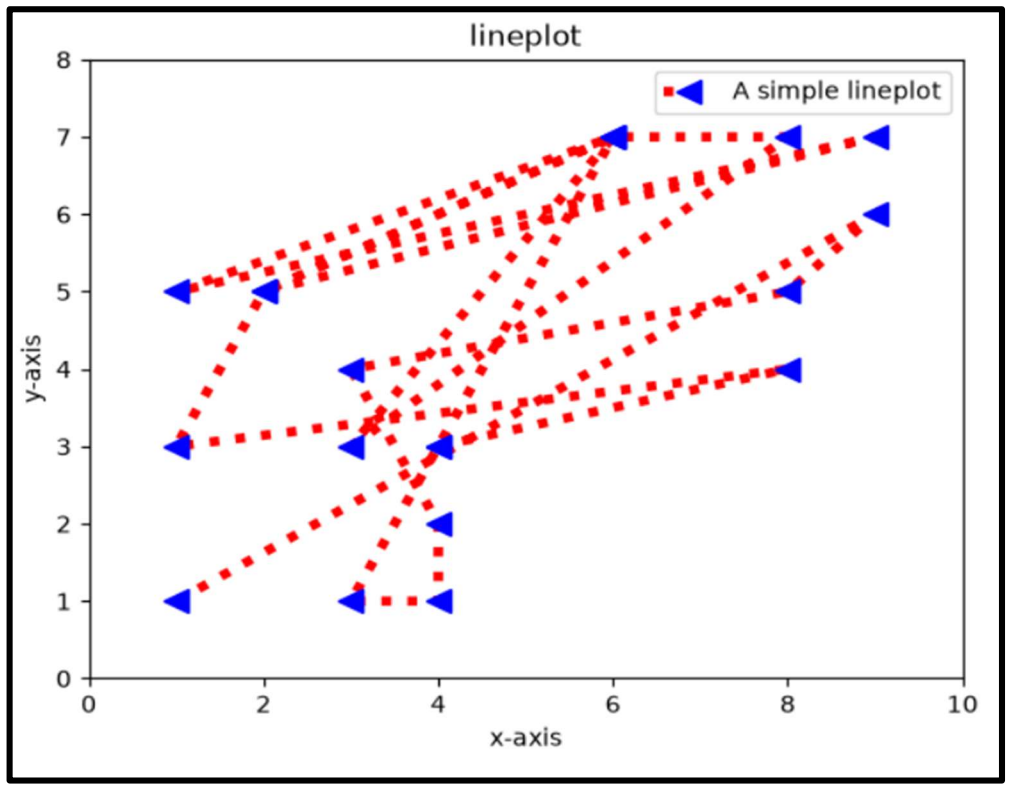
plt.ylabel("y-axis")

# Change x and y limits using axis() :- plt.axis([xmin, xmax, ymin, ymax])

plt.axis([0, 10, 0, 8])

plt.legend(["A simple lineplot"])

plt.show()
```



10. Using Grid method :-

```
x = np.random.randint(low=1, high=10, size=20)

y = np.random.randint(low=1, high=8, size=20)

plt.plot(x, y, color='r', linewidth=4, marker='<', markersize=10, markerfacecolor='blue', markeredgecolor='blue', linestyle=':')

plt.title("lineplot")

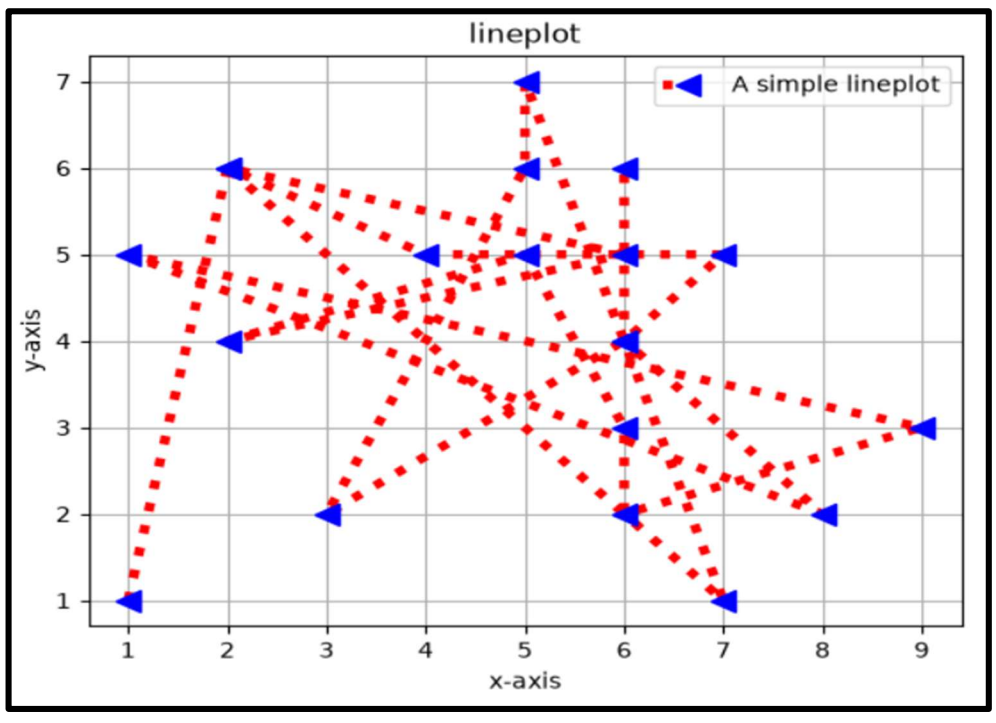
plt.xlabel("x-axis")

plt.ylabel("y-axis")

plt.grid()

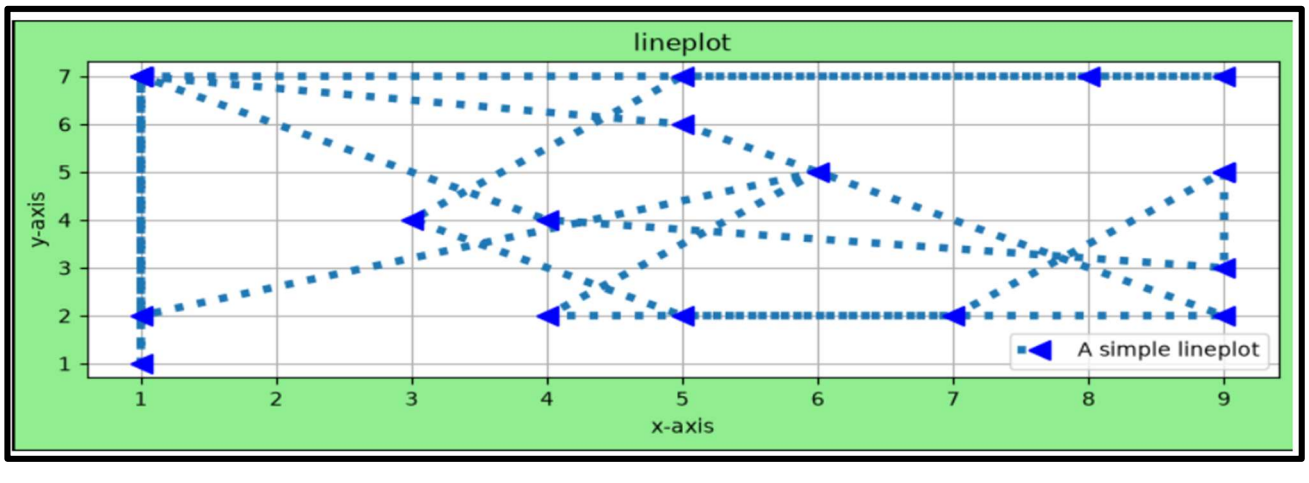
plt.legend(["A simple lineplot"])

plt.show()
```



10. Change Figure Size :-

```
plt.figure(figsize=(10,3), facecolor='lightgreen',edgecolor='black',linewidth=1)
```



Histogram Example :-

```
import matplotlib.pyplot as plt

import pandas as pd

data = pd.read_csv('titanic.csv')

data.head()
```

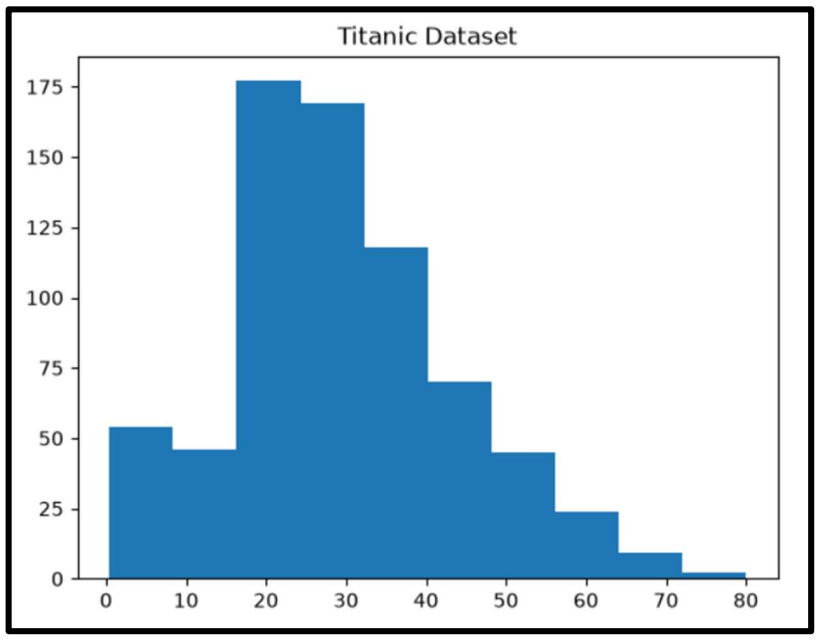
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

1. Plot A Histogram For Age Column.

```
plt.hist(data['Age'])

plt.title("Titanic Dataset")

plt.show()
```



2. Use Bins Parameter.

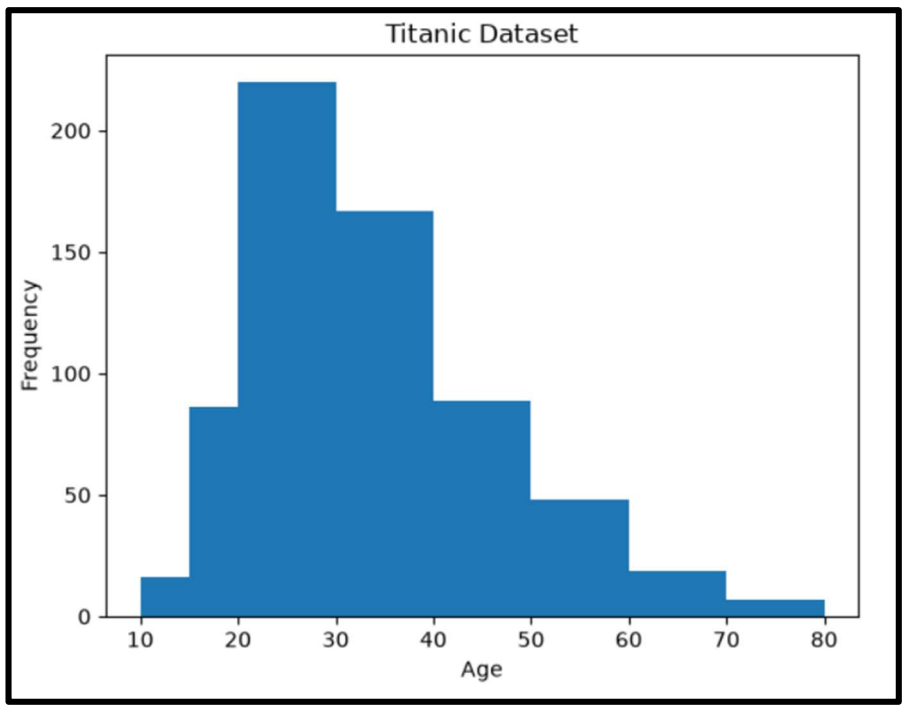
```
plt.hist(data['Age'], bins=[10,15,20,30,40,50,60,70,80])

plt.title("Titanic Dataset")

plt.xlabel("Age")

plt.ylabel("Frequency")

plt.show()
```



3. Set Relative Width of the Bars.

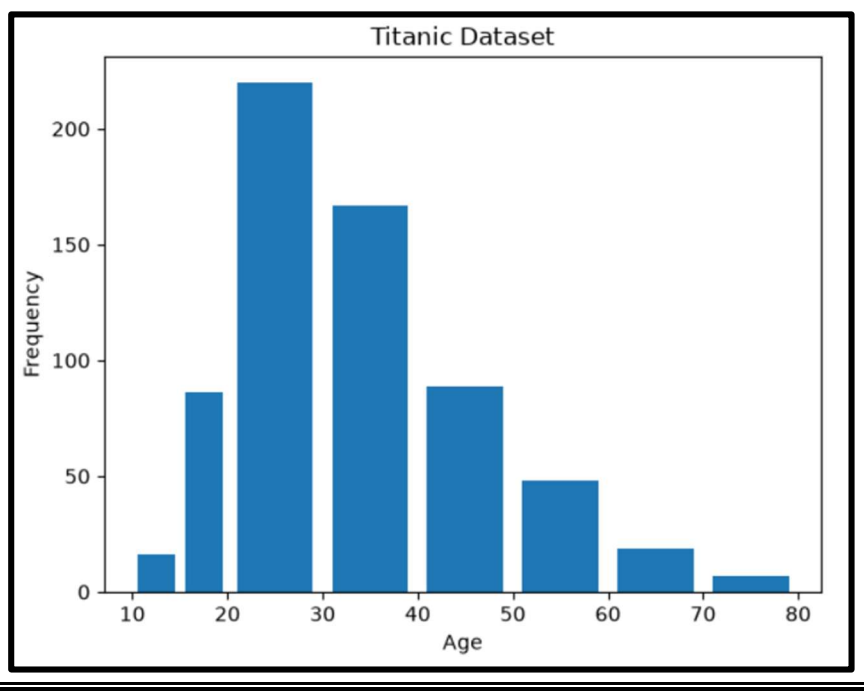
```
plt.hist(data['Age'], bins=[10,15,20,30,40,50,60,70,80], rwidth=0.80)

plt.title("Titanic Dataset")

plt.xlabel("Age")

plt.ylabel("Frequency")

plt.show()
```



4. Change Histtype & Align Bars To Middle.

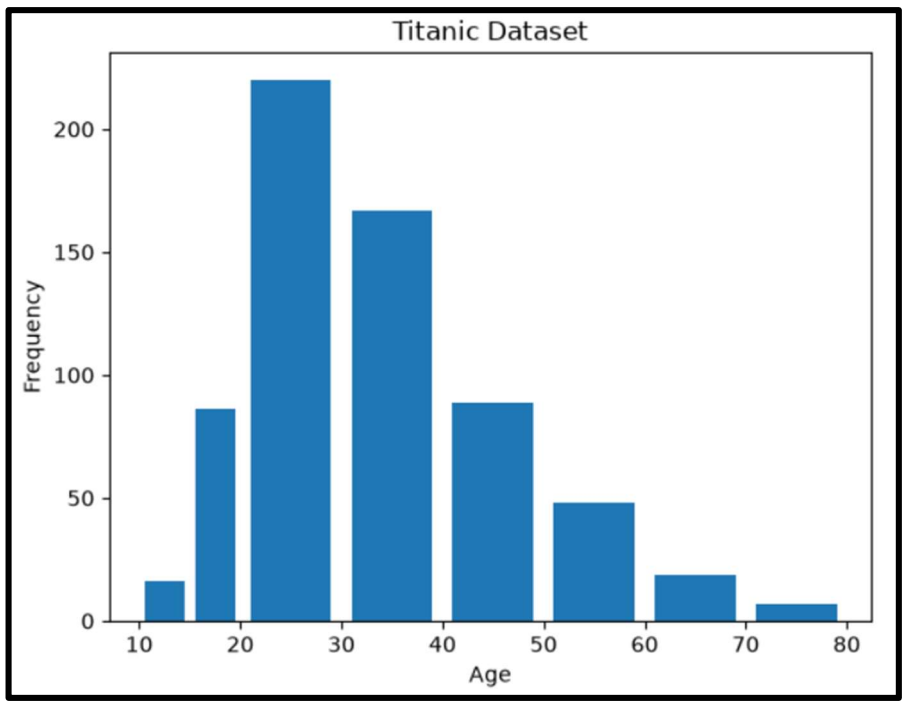
```
plt.hist(data['Age'],bins=[10, 15, 20, 30, 40, 50, 60, 70, 80],rwidth=0.80,histtype='bar',align='mid')

plt.title("Titanic Dataset")

plt.xlabel("Age")

plt.ylabel("Frequency")

plt.show()
```



5. Change Orientation To Horizontal And Change Color of the Bars.

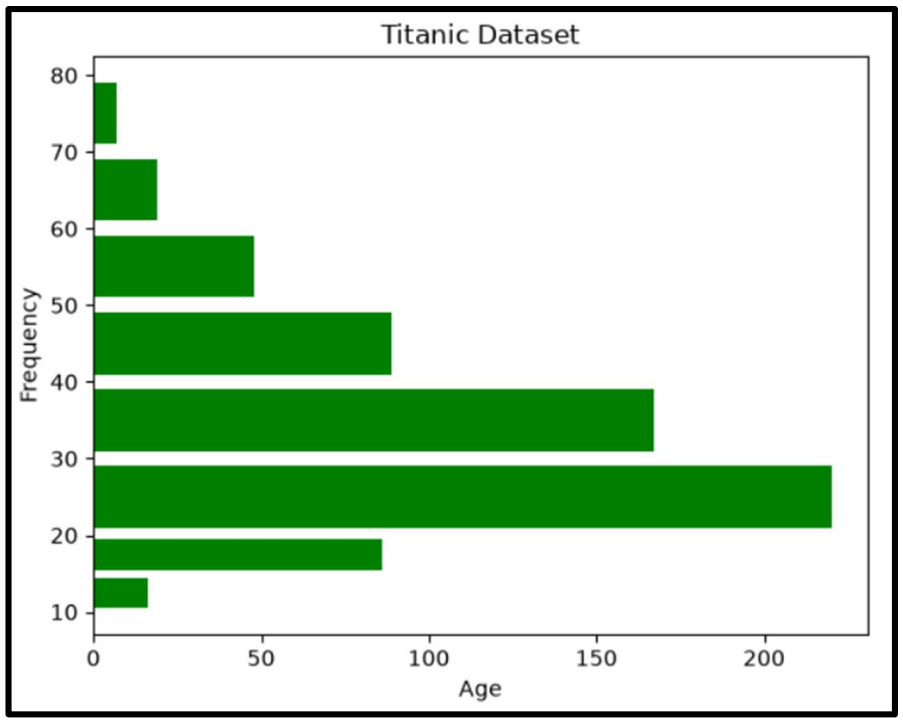
```
plt.hist(data['Age'],bins=[10, 15, 20, 30, 40, 50, 60, 70, 80],rwidth=0.80,histtype='bar',
align='mid',orientation='horizontal',color='green')

plt.title("Titanic Dataset")

plt.xlabel("Age")

plt.ylabel("Frequency")

plt.show()
```



6. Draw Histograms for Both the Subject Scores.

```
import matplotlib.pyplot as plt

Subject1_scores = [1,1,1,2,3,3,3,4,4,4,4,5,5,5,5]

Subject2_scores = [1,1,2,2,3,3,4,4,4,4,5,5,5,5,5]

plt.hist(Subject1_scores, bins=5,rwidth=0.80,histtype='bar',align='mid',orientation='vertical',color='r')

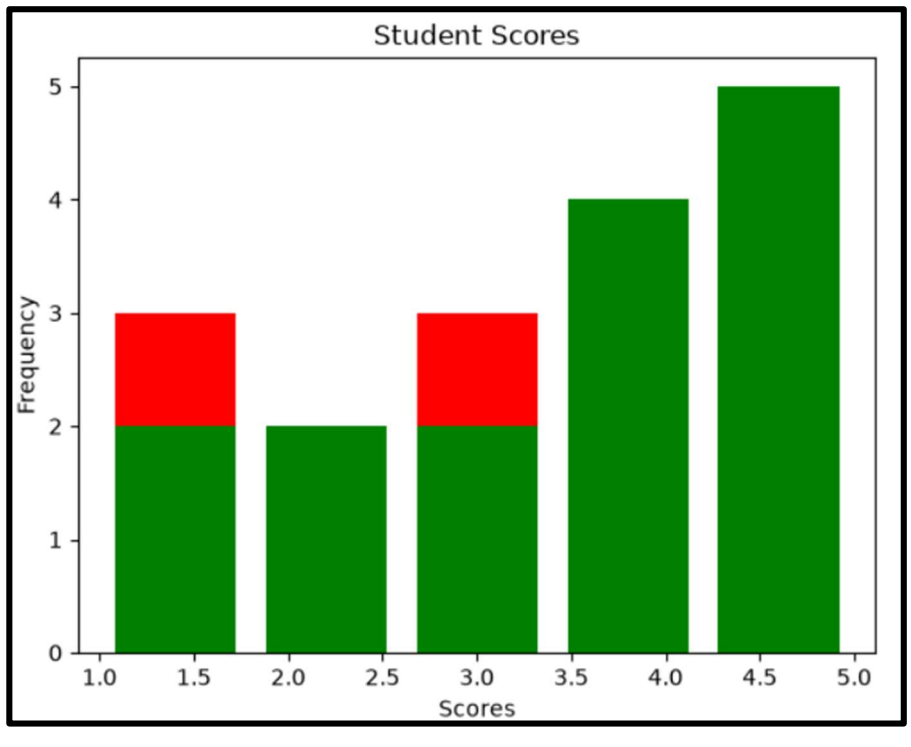
plt.hist(Subject2_scores, bins=5,rwidth=0.80,histtype='bar',align='mid',orientation='vertical',color='g')

plt.title("Student Scores")

plt.xlabel("Scores")

plt.ylabel("Frequency")

plt.show()
```



```
import matplotlib.pyplot as plt

Subject1_scores = [1,1,1,2,3,3,3,4,4,4,4,5,5,5,5]

Subject2_scores = [1,1,2,2,3,3,4,4,4,4,5,5,5,5,5]

plt.hist([Subject1_scores, Subject2_scores],bins=5,rwidth=0.80,histtype='bar',align='mid',orientation='vertical',color=['r', 'b'],label=['Subject 1 scores',
'Subject 2 scores'])

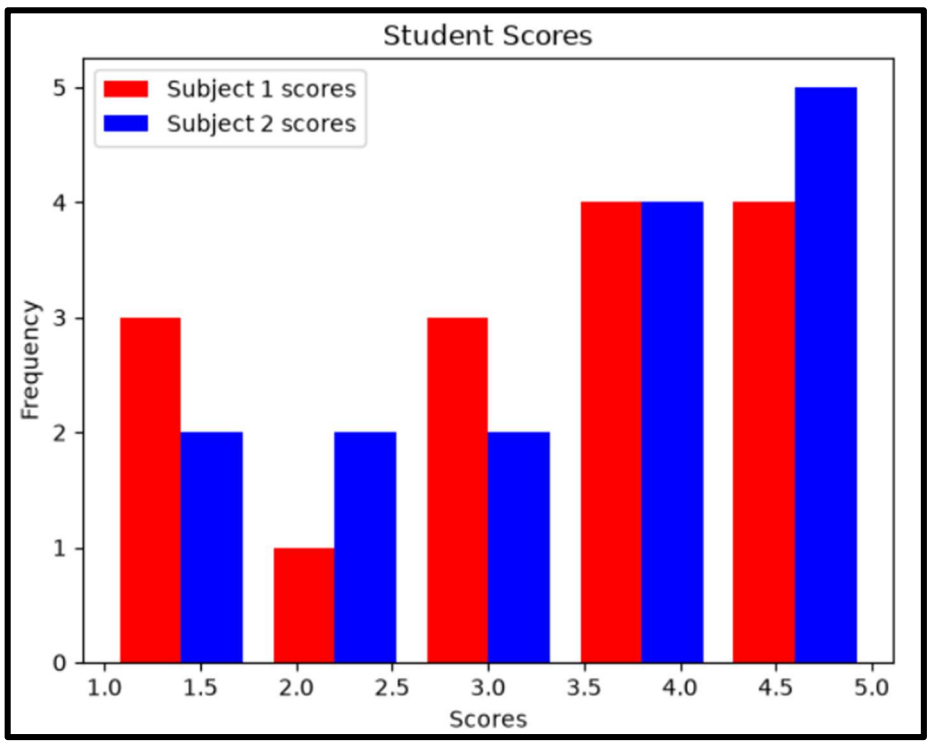
plt.title("Student Scores")

plt.xlabel("Scores")

plt.ylabel("Frequency")

plt.legend()

plt.show()
```



7. Change Limit of X-axis and Y-axis.

```
x_ticks = [x for x in range(1, 6)]

y_ticks = [y for y in range(1, 6)]

plt.hist([Subject1_scores, Subject2_scores], bins=5, rwidth=0.80, histtype='bar', align='mid', orientation='vertical', color=['r', 'b'], label=['Subject 1
scores', 'Subject 2 scores'])

plt.xlim(0, 6)    # X-axis limit

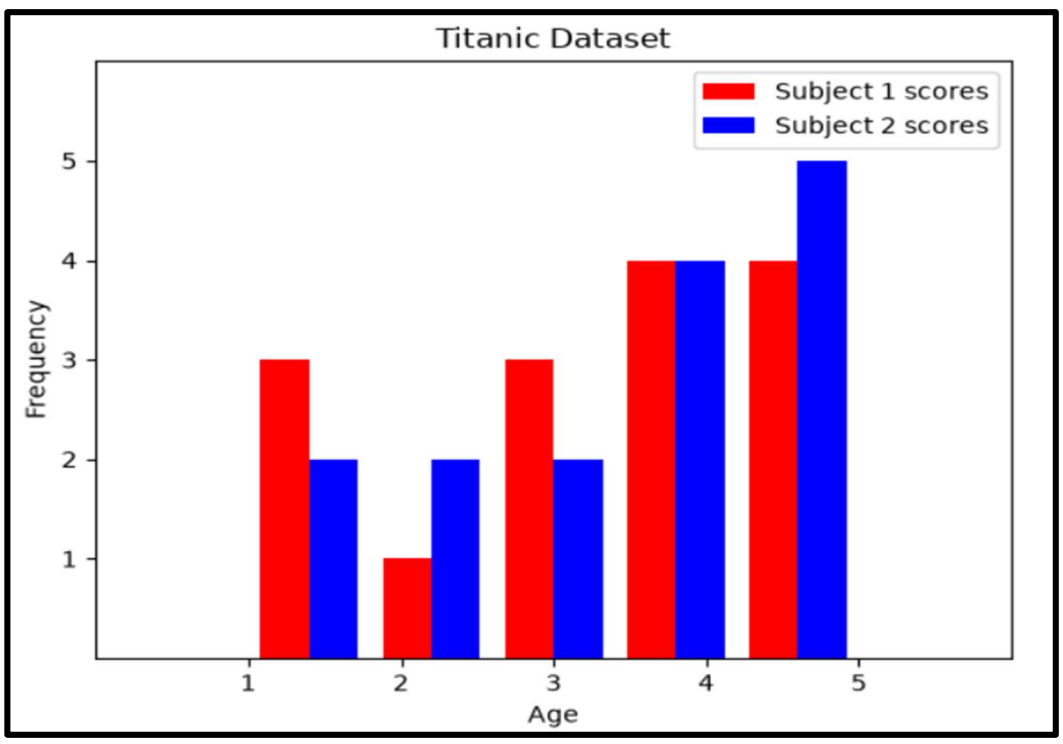
plt.ylim(0, 6)    # Y-axis limit

plt.xticks(x_ticks)

plt.yticks(y_ticks)

plt.legend()

plt.show()
```



8. Change Figure Size And Color.

```
Subject1_scores = [1,1,1,2,3,3,3,4,4,4,4,5,5,5,5]

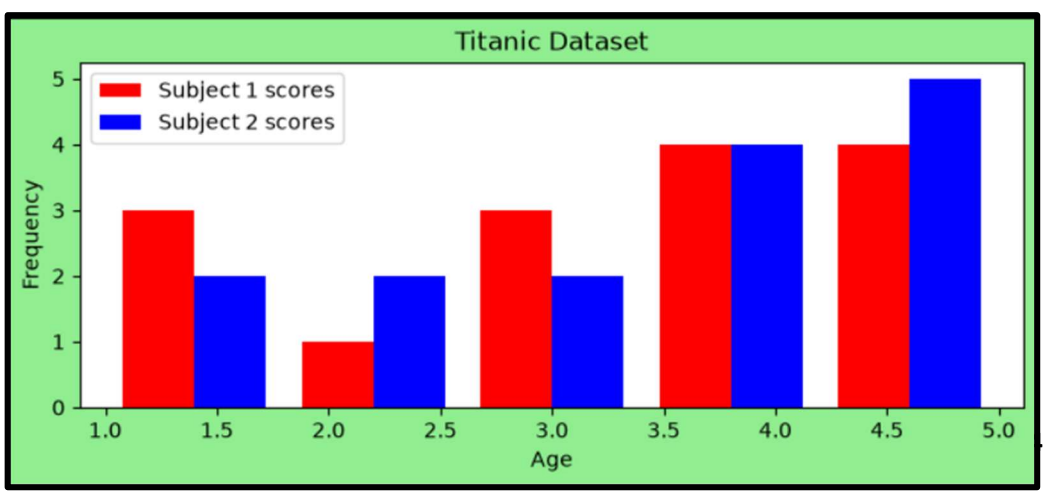
Subject2_scores = [1,1,2,2,3,3,4,4,4,4,5,5,5,5,5]

plt.figure(figsize=(8, 3), facecolor='lightgreen', edgecolor='black', linewidth=2 )

plt.hist([Subject1_scores, Subject2_scores], bins=5, rwidth=0.80, histtype='bar', align='mid', orientation='vertical', color=['r', 'b'], label=['Subject 1
scores', 'Subject 2 scores'] )

plt.legend()

plt.show()
```



Bar Chart Example :-

```
import matplotlib.pyplot as plt

import pandas as pd

grades = ['A','B','C','D']

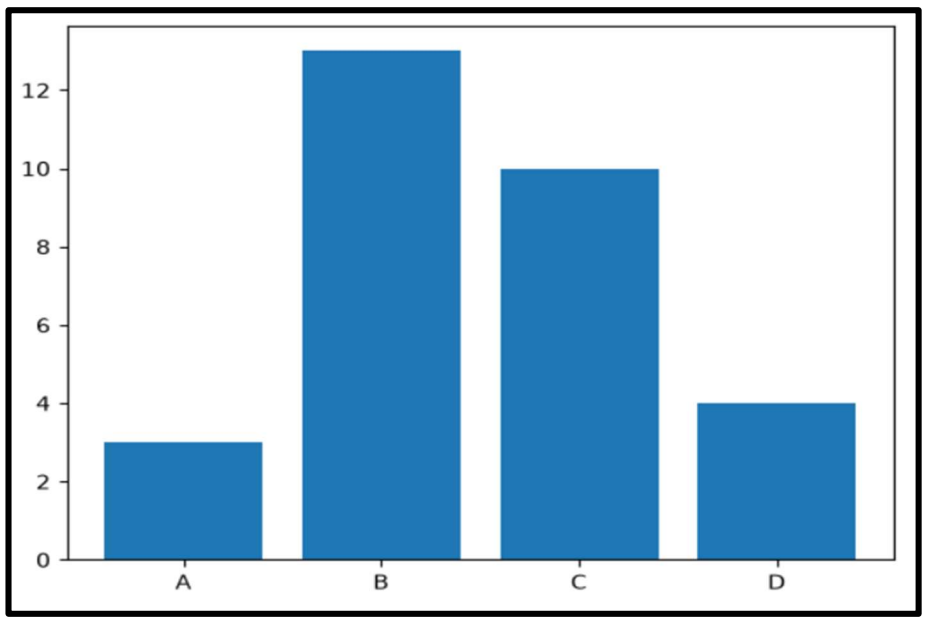
test1 = [3,13,10,4]

test2 = [5,24,6,5]
```

1. Draw A Bar Plot

```
plt.bar(grades,test1)

plt.show()
```



2. Give Title, Xlabel and Ylabel

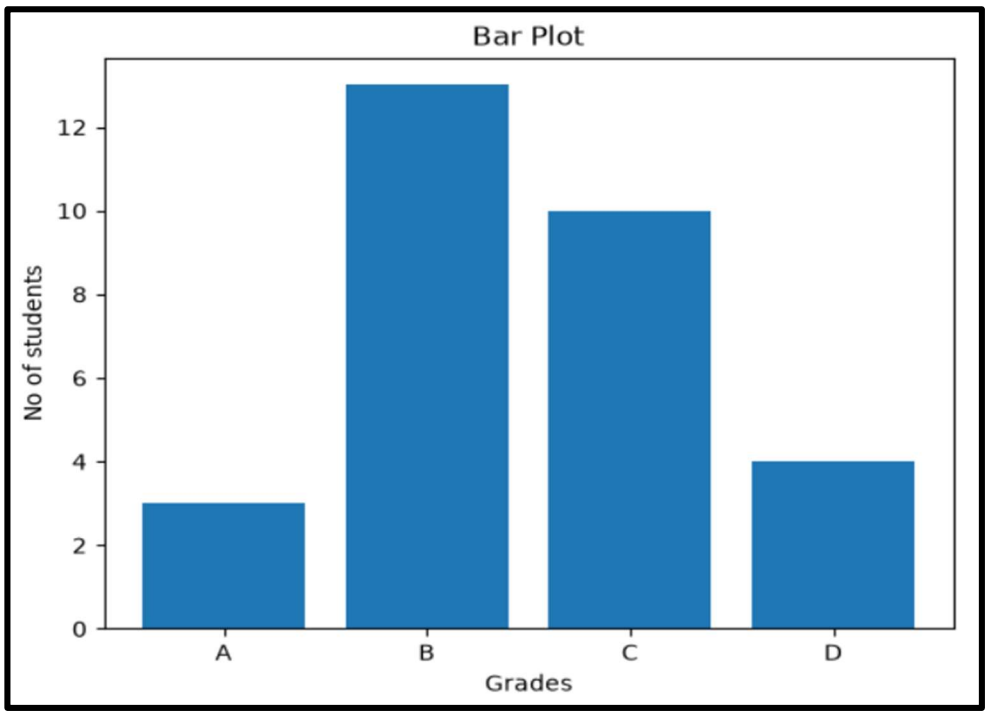
```
plt.bar(grades,test1)

plt.title("Bar Plot")

plt.xlabel("Grades")

plt.ylabel("No of students")

plt.show()
```



3. Change FontSize of Title, Xlabel, and Ylabel

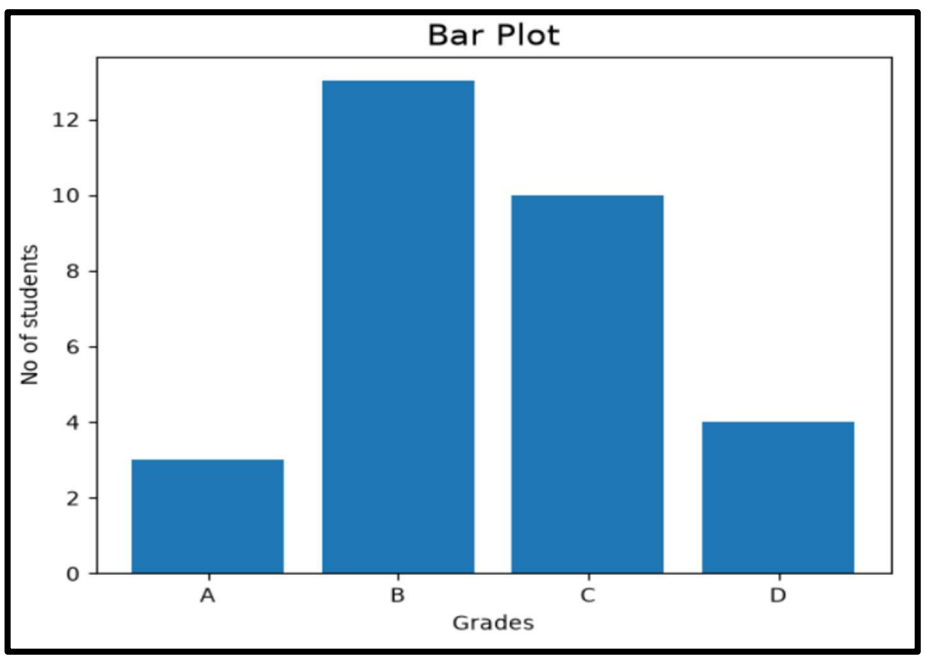
```
plt.bar(grades,test1)

plt.title("Bar Plot",fontsize=15)

plt.xlabel("Grades",fontsize=10)

plt.ylabel("No of students",fontsize=10)

plt.show()
```



4. Set Width of The Bars & Align Bars To Center

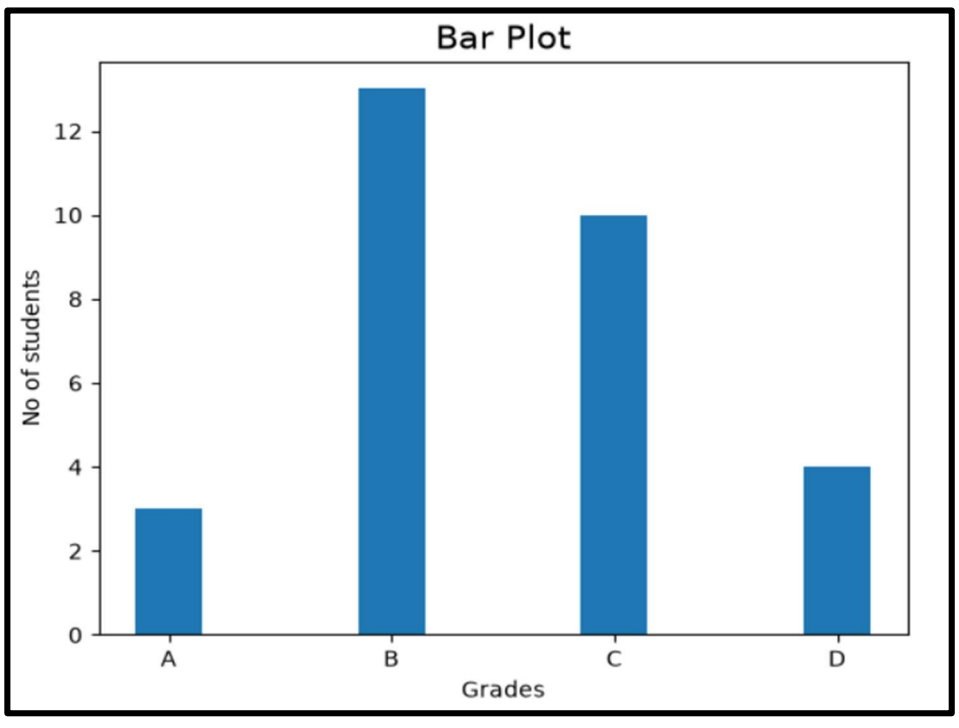
```
plt.bar(grades, test1, width=0.3, align='center')

plt.title("Bar Plot", fontsize=15)

plt.xlabel("Grades", fontsize=10)

plt.ylabel("No of students", fontsize=10)

plt.show()
```



5. Change Color of the Bars & Also Change Edges Color.

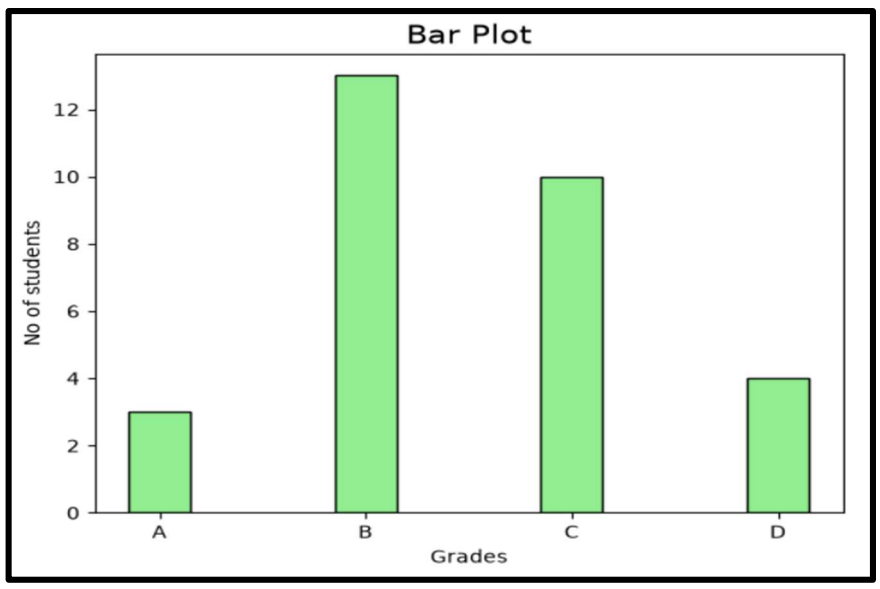
```
plt.bar(grades, test1, width=0.3, align='center',color='lightgreen', edgecolor='black')

plt.title("Bar Plot", fontsize=15)

plt.xlabel("Grades", fontsize=10)

plt.ylabel("No of students", fontsize=10)

plt.show()
```



6. Change Line Styles & Line Width of The Bars.

```
# linestyle='-'    # Solid

# linestyle='--'   # Dashed

# linestyle=':'    # Dotted

# linestyle='-.'   # Dash-dot

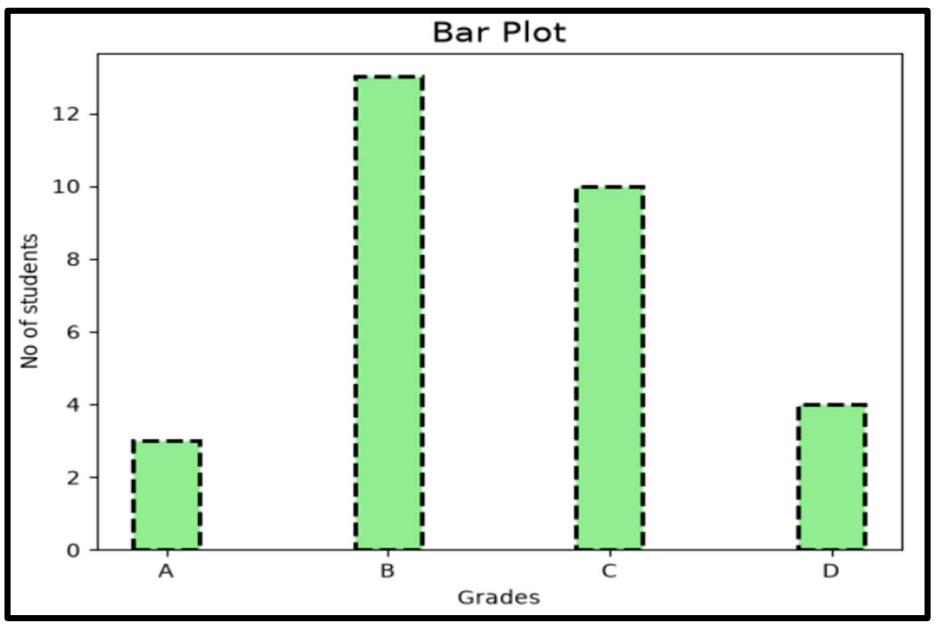
plt.bar(grades,test1,width=0.3,align='center',color='lightgreen',edgecolor='black',linestyle='--',linewidth=2)

plt.title("Bar Plot", fontsize=15)

plt.xlabel("Grades", fontsize=10)

plt.ylabel("No of students", fontsize=10)

plt.show()
```



7. Draw BarPlots For Test1 and Test2 And Add Legend.

```
plt.bar(grades, test1, width=0.3, align='center',color='lightgreen', edgecolor='black',label='Test 1')

plt.bar(grades, test2, width=0.3, align='edge',color='skyblue', edgecolor='black',label='Test 2')

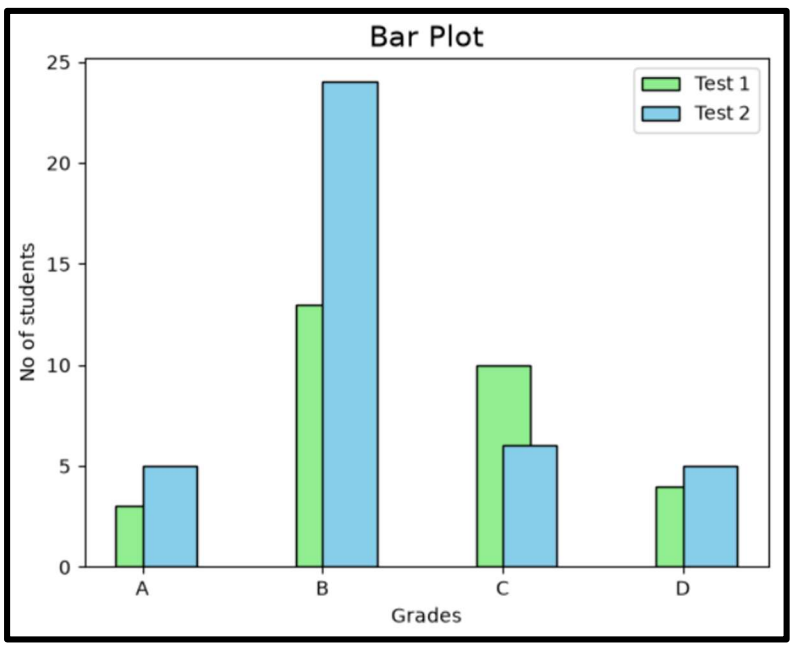
plt.title("Bar Plot", fontsize=15)

plt.xlabel("Grades", fontsize=10)

plt.ylabel("No of students", fontsize=10)

plt.legend()

plt.show()
```



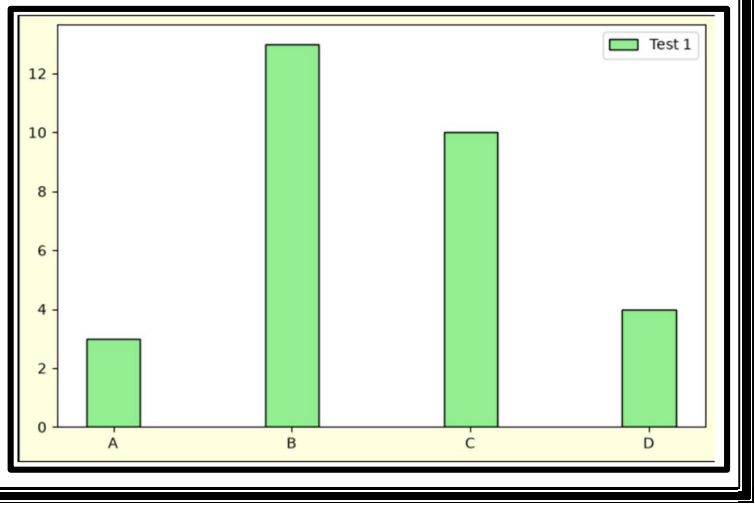
8. Figure Size and Colors.

```
plt.figure(figsize=(8, 5), facecolor='lightyellow',edgecolor='black',linewidth=2)

plt.bar(grades, test1, width=0.3, align='center',color='lightgreen', edgecolor='black',label='Test 1')

plt.legend()

plt.show()
```



9. Change Plot Style & Save the Graph.

```
# Change plot style

plt.style.use('ggplot')

plt.bar(grades,test1,width=0.3,align='center',color='lightgreen',edgecolor='black',label='Test 1')

plt.title("Bar Plot", fontsize=15)

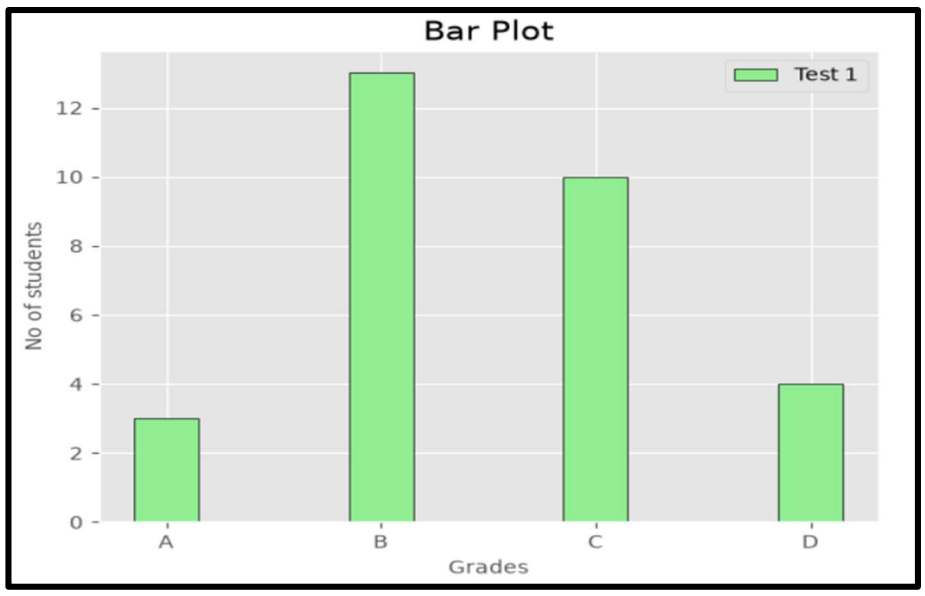
plt.xlabel("Grades", fontsize=10)

plt.ylabel("No of students", fontsize=10)

plt.legend()

plt.savefig("bar_plot.png")   # Save the graph

plt.show()
```



Pie Chart Example :-

```
import matplotlib.pyplot as plt
```

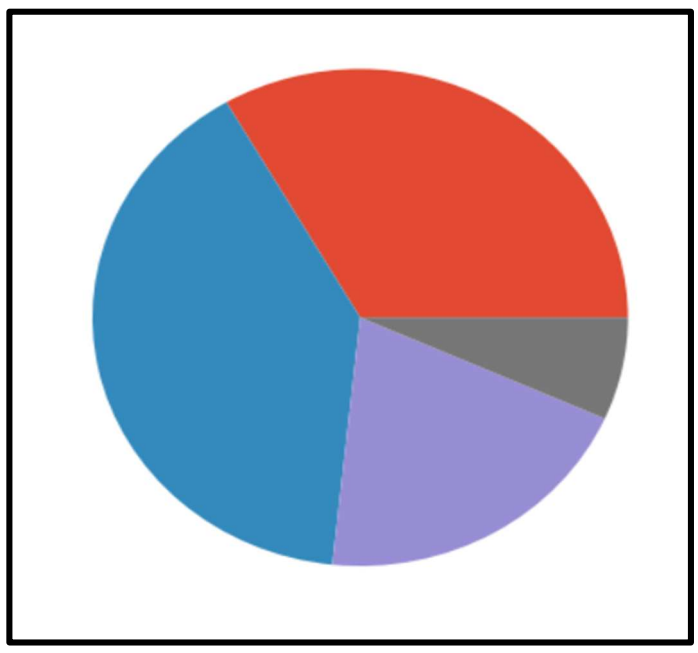
1. Draw A Pie Chart

```
movie = ['comedy','Action','Romance','Drama']

votes = [5,6,3,1]

plt.pie(votes)

plt.show()
```



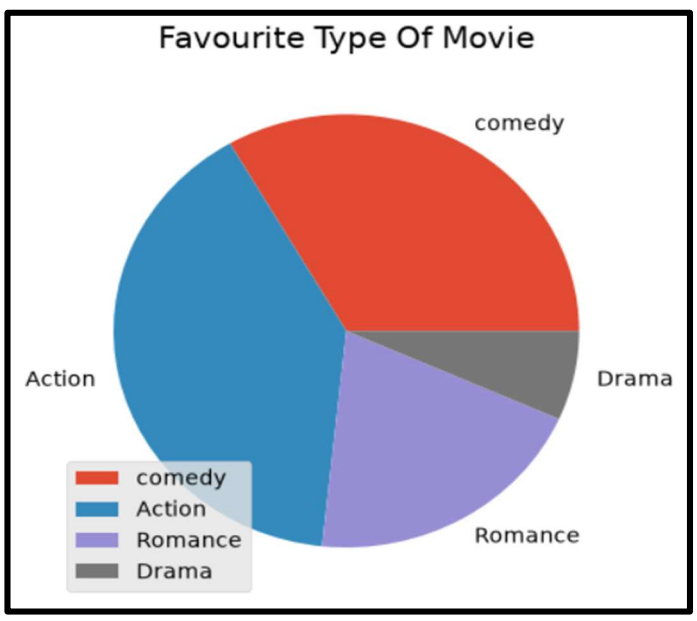
2. Give Title, Label And Legend

```
plt.title("Favorite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1)

plt.legend()

plt.show()
```



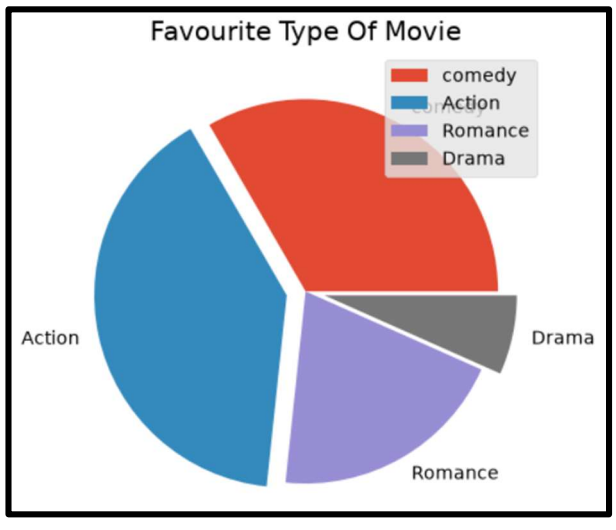
3. Pull Out Action and Drama Wedges

```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1])

plt.legend()

plt.show()
```



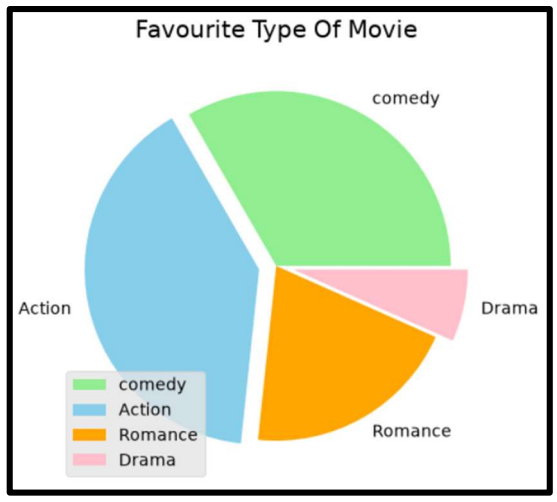
4. Set the color of each wedge

```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'])

plt.legend(loc=3)

plt.show()
```



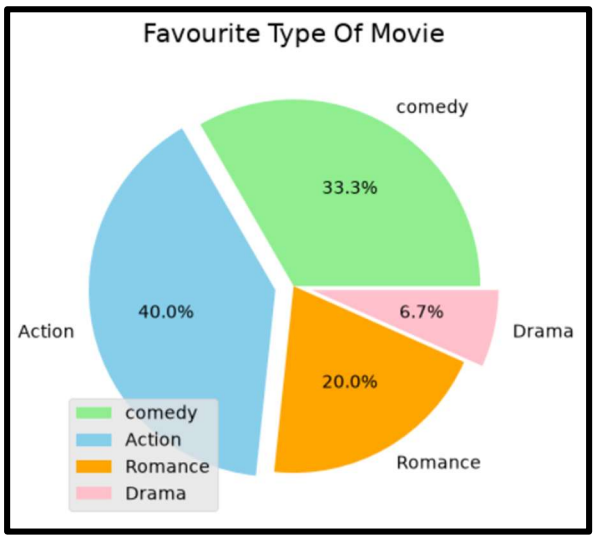
5. Display Percentage & Change Distance Inside Each Wedge

```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'],autopct='%1.1f%%',pctdistance=0.6)

plt.legend(loc=3)

plt.show()
```



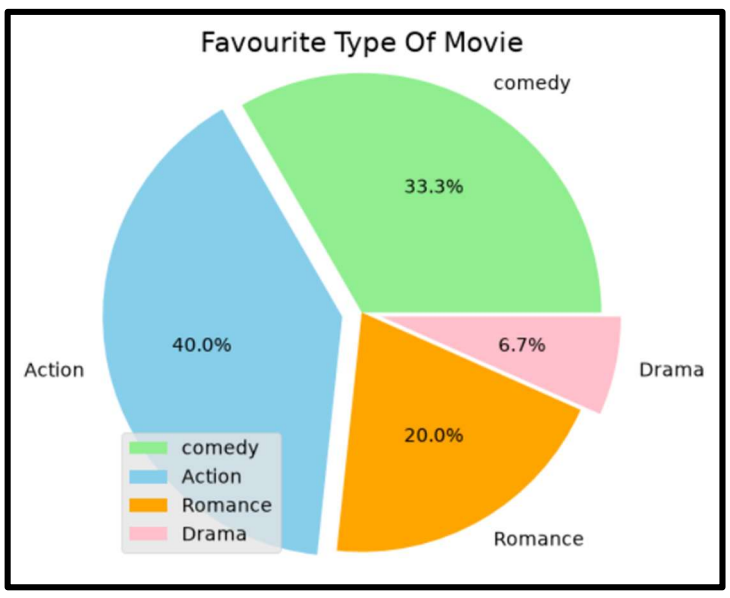
6. Change Radius Of The Pie Chart

```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'], autopct='%1.1f%%', pctdistance=0.6,
radius=1.2)

plt.legend(loc=3)

plt.show()
```



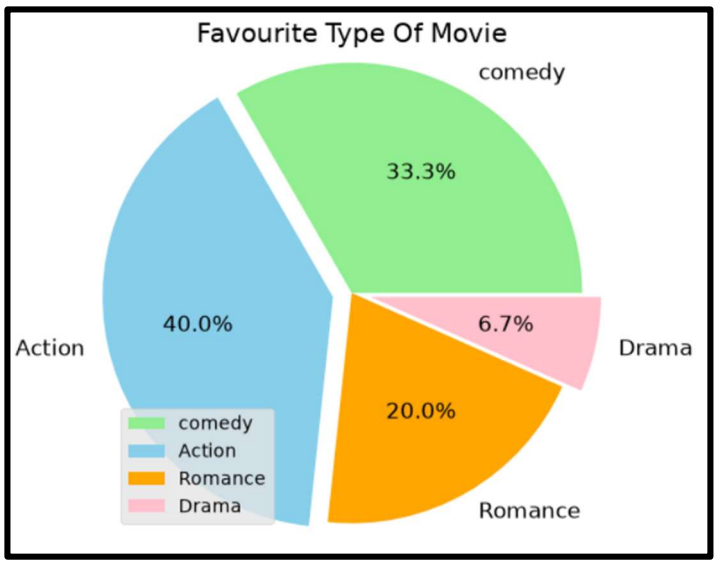
7. Set Label Size Of the Pie Chart

```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'],autopct='%1.1f%%',pctdistance=0.6,
textprops={'fontsize': 12})

plt.legend(loc=3)

plt.show()
```



8. Set border of wedges

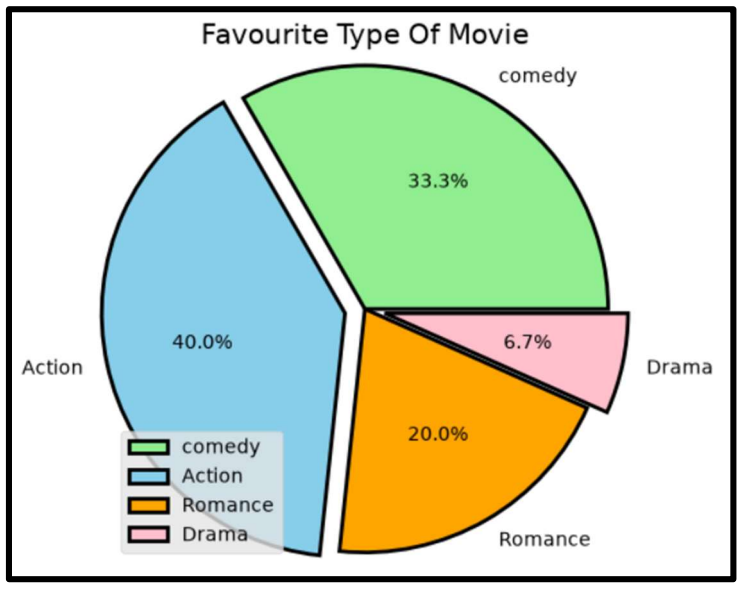
```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'],autopct='%1.1f%%',pctdistance=0.6,

wedgeprops={'edgecolor': 'black', 'linewidth': 2})

plt.legend(loc=3)

plt.show()
```



9. Add A Shadow To the pie chart

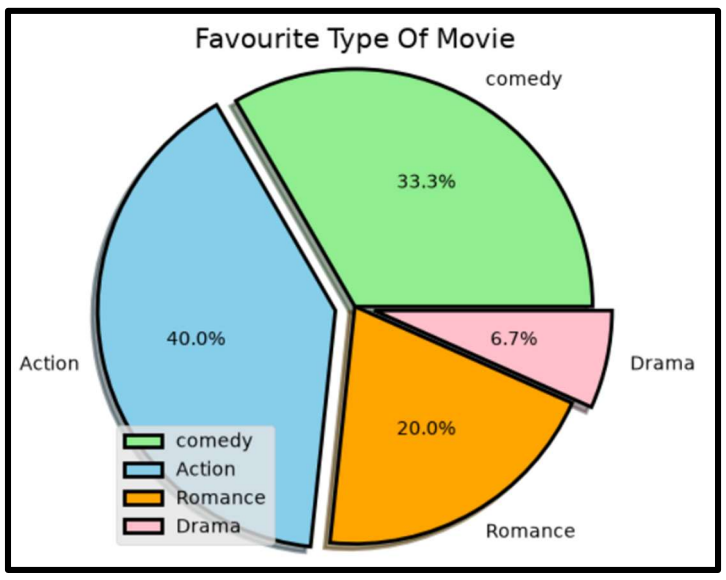
```
plt.title("Favourite Type Of Movie")

plt.pie(votes, labels=movie,labeldistance=1.1,explode=[0,0.1,0,0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'],autopct='%1.1f%%',pctdistance=0.6,

wedgeprops={'edgecolor': 'black', 'linewidth': 2},shadow=True)

plt.legend(loc=3)

plt.show()
```



10. Change Figure size & color

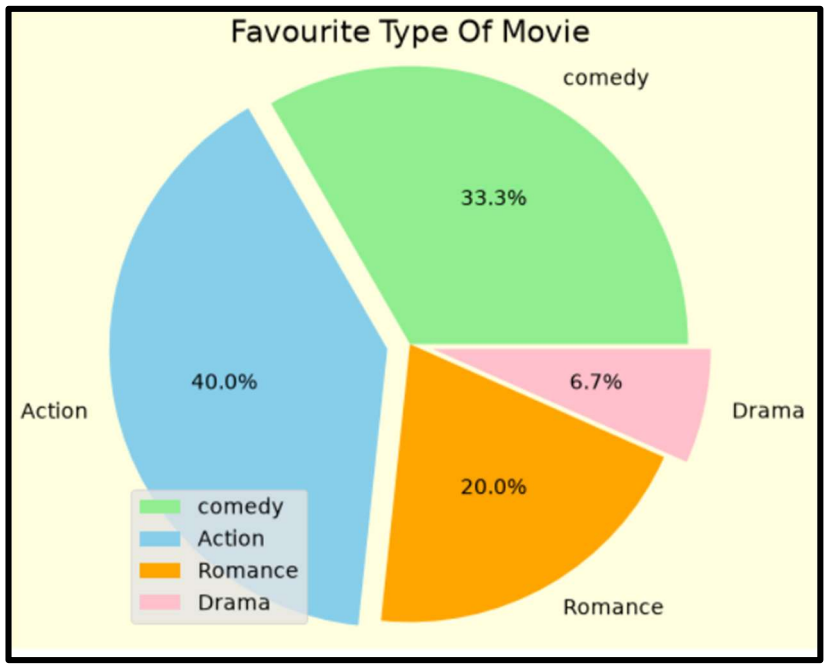
```
plt.figure(figsize=(8, 5), facecolor='lightyellow')

plt.title("Favourite Type Of Movie")

plt.pie(votes,labels=movie,labeldistance=1.1,explode=[0, 0.1, 0, 0.1],colors=['lightgreen', 'skyblue', 'orange', 'pink'],autopct='%1.1f%%', pctdistance=0.6)

plt.legend(loc=3)

plt.show()
```



Scatter Chart :-

Scatter plot shows how two variables are related by representing each data point as a dot.

Example :-

```
import matplotlib.pyplot as plt
```

1. Draw A Scatter Plot With Title, Xlabel And Ylabel

```
temp = [14.2, 16.4, 11.9, 15.2, 18.5, 22.1, 19.4]

icecream_sales = [215, 325, 185, 332, 406, 522, 412]

tea_sales = [80, 65, 89, 70, 56, 45, 43]

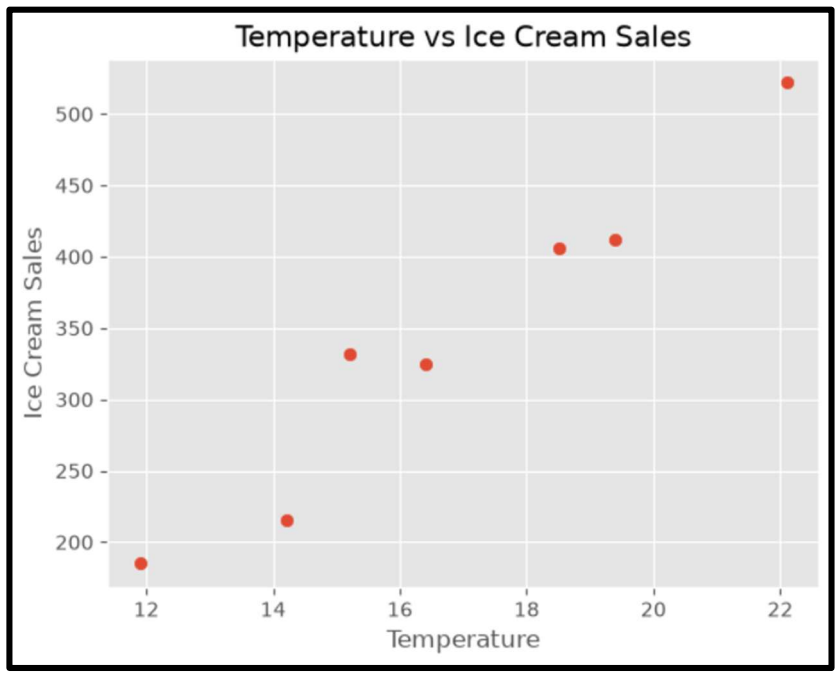
plt.scatter(temp, icecream_sales)

plt.title("Temperature vs Ice Cream Sales")

plt.xlabel("Temperature")

plt.ylabel("Ice Cream Sales")

plt.show()
```



2. Change Size & Color of The Dots

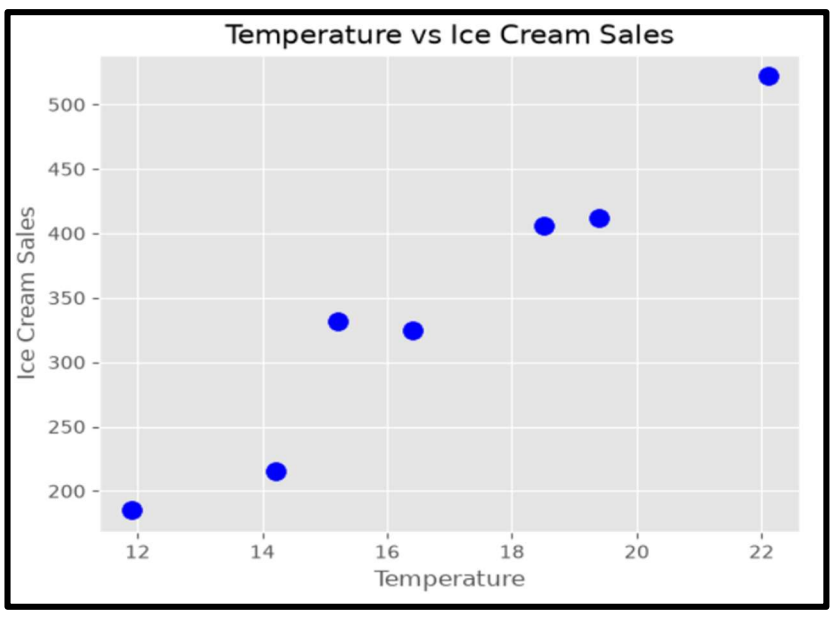
```
plt.scatter(temp, icecream_sales,s=100,c='blue')

plt.title("Temperature vs Ice Cream Sales")

plt.xlabel("Temperature")

plt.ylabel("Ice Cream Sales")

plt.show()
```



3. Change the marker

```
plt.scatter(temp, icecream_sales,s=100,c='blue',marker='*')

plt.title("Temperature vs Ice Cream Sales")

plt.xlabel("Temperature")

plt.ylabel("Ice Cream Sales")

plt.show()
```

```
# marker='o'   # Circle

# marker='*'   # Star

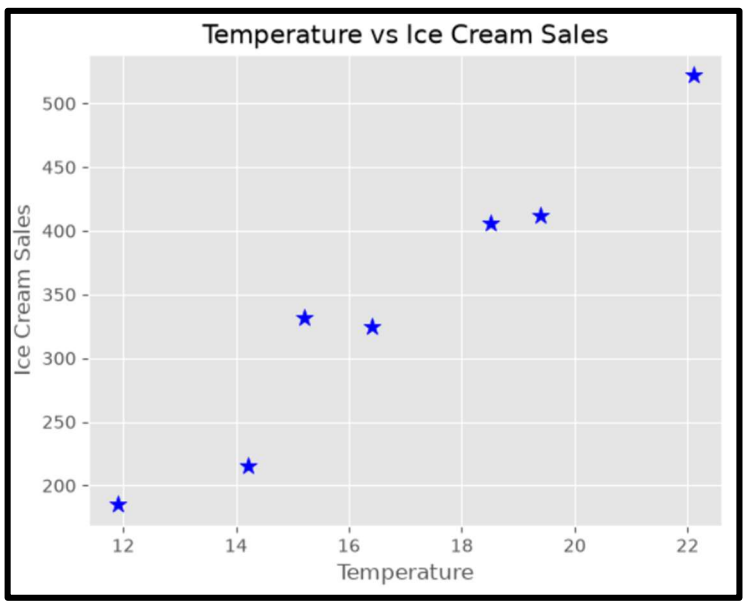
# marker='s'   # Square

# marker='^'   # Triangle

# marker='D'   # Diamond

# marker='+'   # Plus

# marker='x'   # X
```

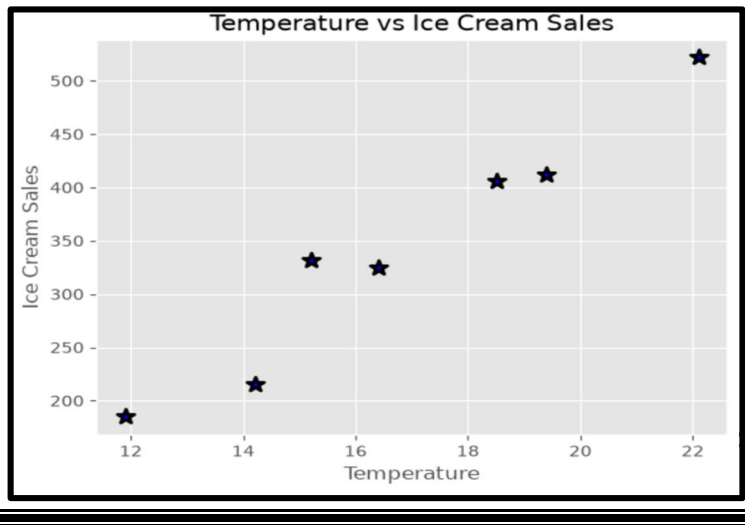


4. Change the edge color of the marker also change line width

```
plt.scatter(temp, icecream_sales,s=100,c='blue',marker='*',

edgecolors='black',linewidths=2)

plt.show()
```



5. Change Alpha Parameter

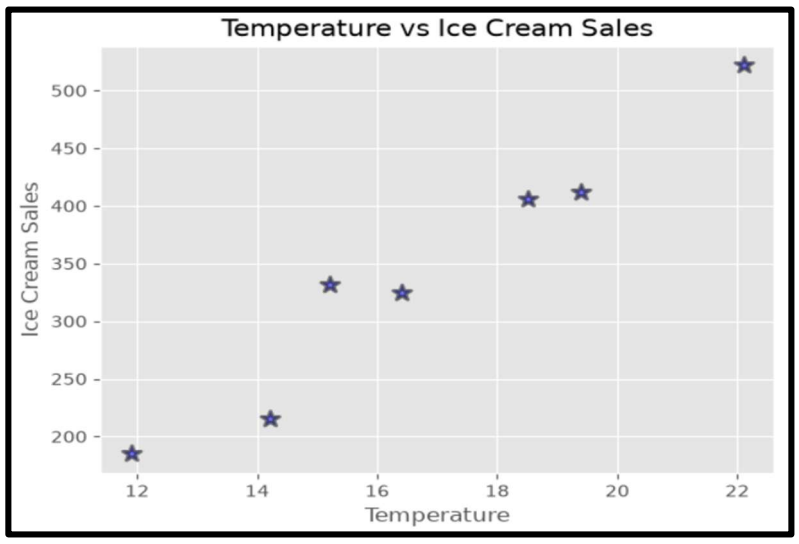
```
plt.scatter(temp, icecream_sales,s=100,c='blue',marker='*',edgecolors='black',linewidths=2, alpha=0.5)

plt.title("Temperature vs Ice Cream Sales")

plt.xlabel("Temperature")

plt.ylabel("Ice Cream Sales")

plt.show()
```



6. Set Different Colors For Different Dots

```
color = [x for x in range(11, 18)]

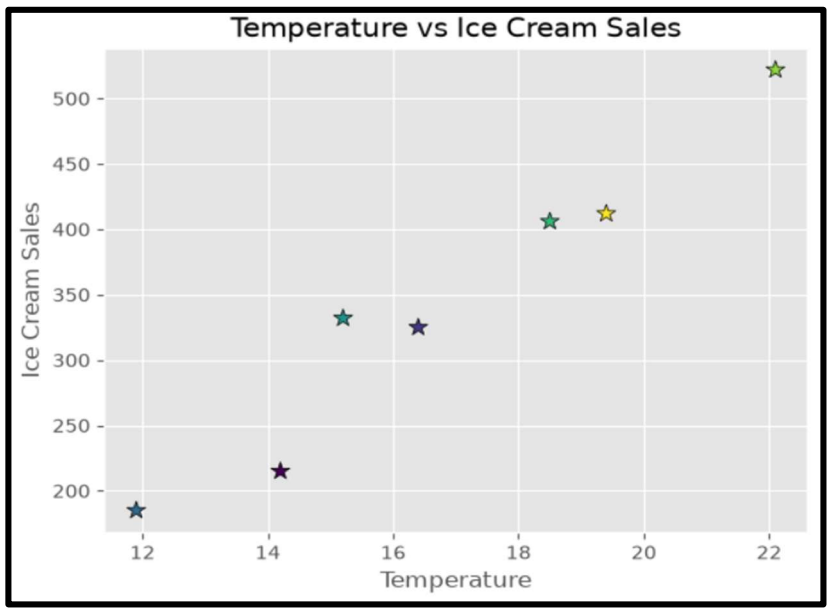
plt.scatter(temp,icecream_sales,s=100,c=color,marker='*',edgecolors='black')

plt.title("Temperature vs Ice Cream Sales")

plt.xlabel("Temperature")

plt.ylabel("Ice Cream Sales")

plt.show()
```



7. Add Color Bar

```
color = [x for x in range(11, 18)]

plt.scatter(temp,icecream_sales,s=100,c=color,marker='o',edgecolors='k',linewidths=2,alpha=1,cmap='Oranges')

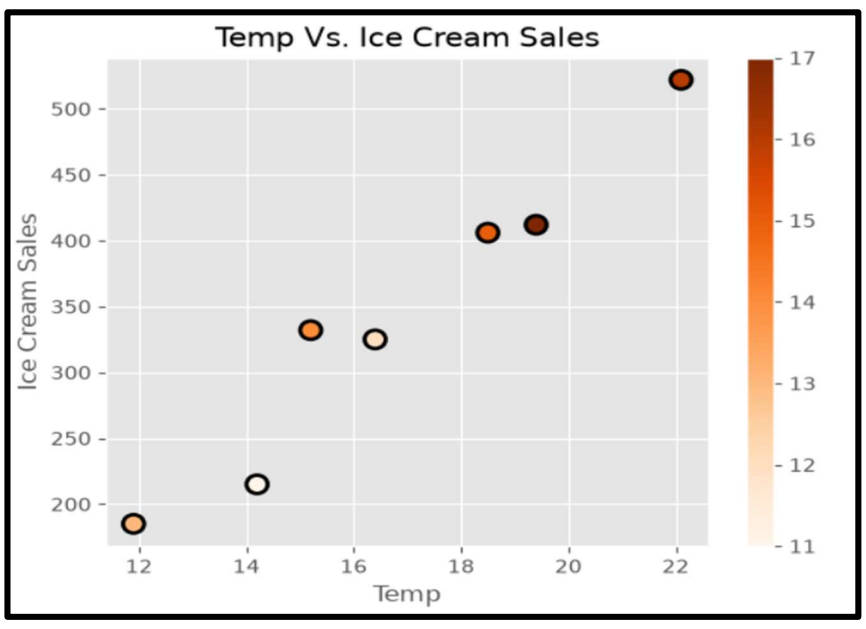
plt.title("Temp Vs. Ice Cream Sales")

plt.xlabel("Temp")

plt.ylabel("Ice Cream Sales")

plt.colorbar()

plt.show()
```



8. Customize Limit of colorbar

```
color = [x for x in range(11, 18)]

plt.scatter(temp,icecream_sales,s=100,c=color,marker='o',edgecolors='k',linewidths=2,alpha=1,cmap='Oranges',vmin=11,vmax=17)

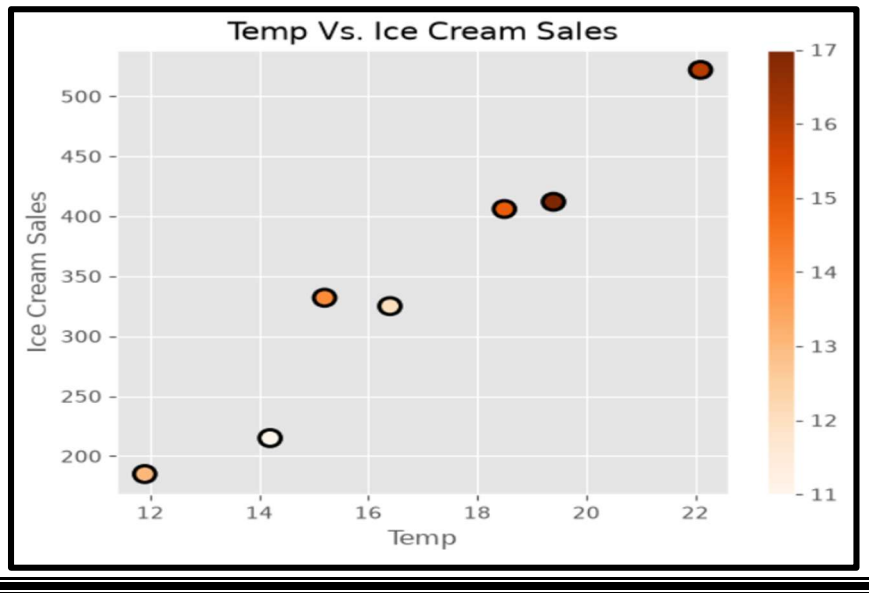
plt.title("Temp Vs. Ice Cream Sales")

plt.xlabel("Temp")

plt.ylabel("Ice Cream Sales")

plt.colorbar()

plt.show()
```



9. Draw Two Scatter Plots in one figure.

```
import matplotlib.pyplot as plt

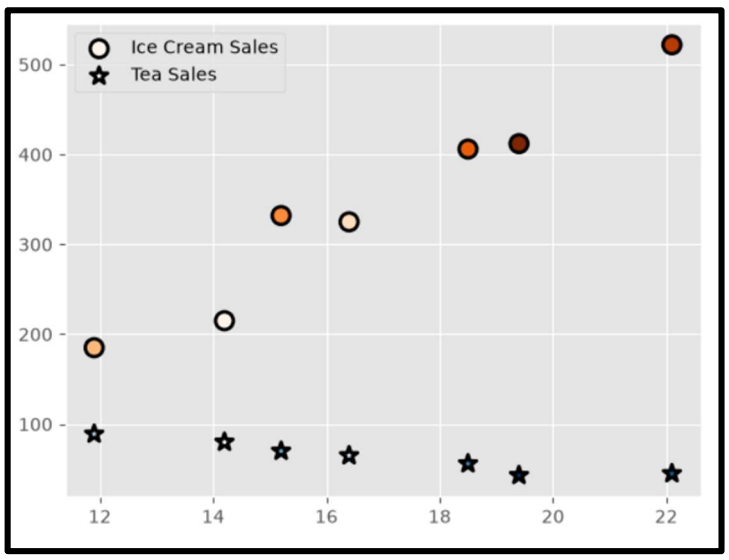
color = [x for x in range(11, 18)]

plt.scatter(temp, icecream_sales,s=100,c=color,marker='o',edgecolors='k',linewidths=2,alpha=1,cmap='Oranges',label='Ice Cream Sales')

plt.scatter(temp, tea_sales,s=100,c=color,marker='*',edgecolors='k',linewidths=2,alpha=1,cmap='Blues',label='Tea Sales')

plt.legend()

plt.show()
```



10. Change Figure Size & Color.

```
import matplotlib.pyplot as plt

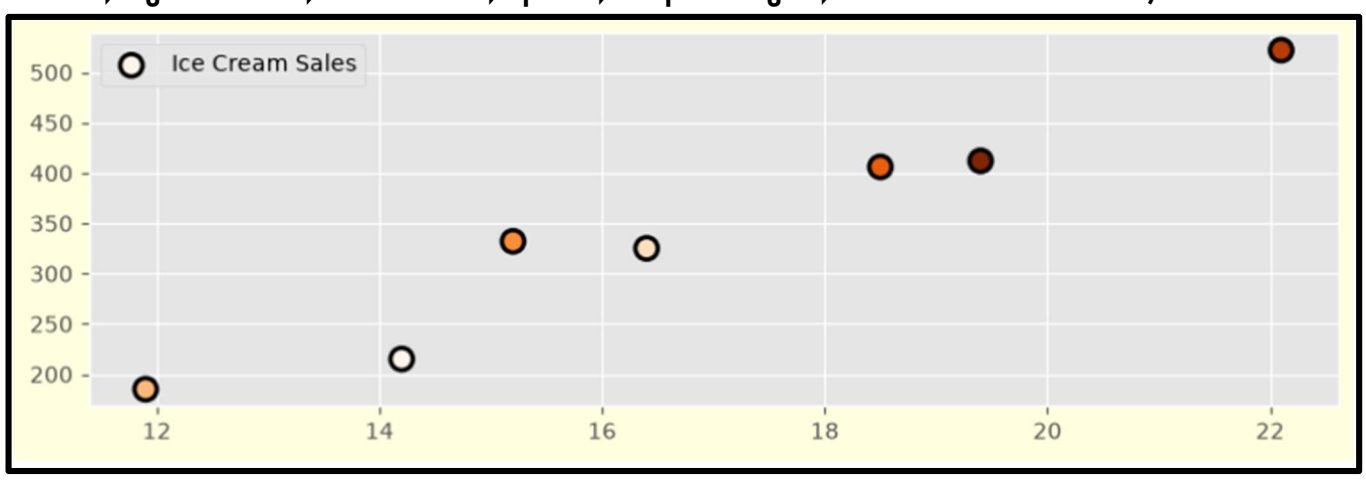
color = [x for x in range(11, 18)]

plt.figure(figsize=(10, 3), facecolor='lightyellow')

plt.scatter(temp, icecream_sales,s=100,c=color,marker='o',edgecolors='k',linewidths=2,alpha=1,cmap='Oranges',label='Ice Cream Sales')

plt.legend()

plt.show()
```



Subplot Chart :-

Example :-

```
import matplotlib.pyplot as plt

import pandas as pd

data=pd.read_csv('iris_code.csv')

data.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

1. Draw Two Plots Side By Side [Histogram of SepalLength & PetalLength]

```
plt.suptitle("Iris Dataset")

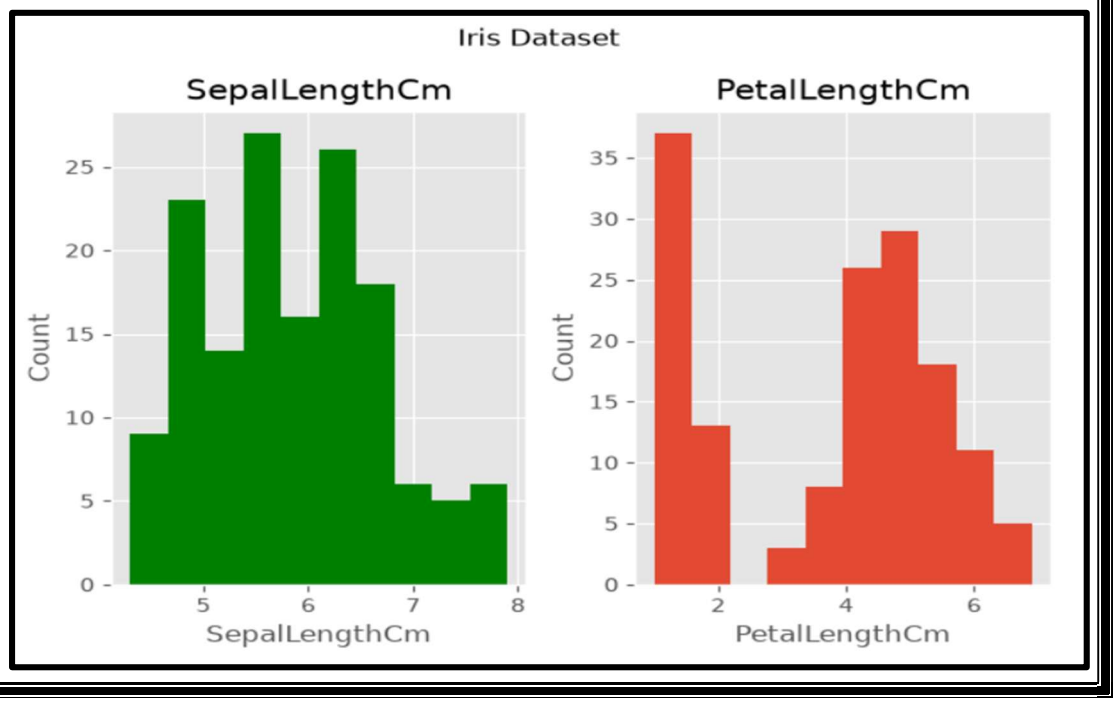
plt.subplot(1, 2, 1)

plt.title("SepalLengthCm")

plt.hist(data.SepalLengthCm,color="green")

plt.xlabel("SepalLengthCm")

plt.ylabel("Count")
```



```
plt.subplot(1, 2, 2)

plt.title("PetalLengthCm")

plt.hist(data.PetalLengthCm)

plt.xlabel("PetalLengthCm")

plt.ylabel("Count")

plt.tight_layout()

plt.show()
```

2. Draw Two Plots Top of Each Other.

```
plt.suptitle("Iris Dataset")

plt.subplot(2, 1, 1)

plt.title("SepalLengthCm")

plt.hist(data.SepalLengthCm,color="green")

plt.xlabel("SepalLengthCm")

plt.ylabel("Count")


plt.subplot(2, 1, 2)

plt.title("PetalLengthCm")

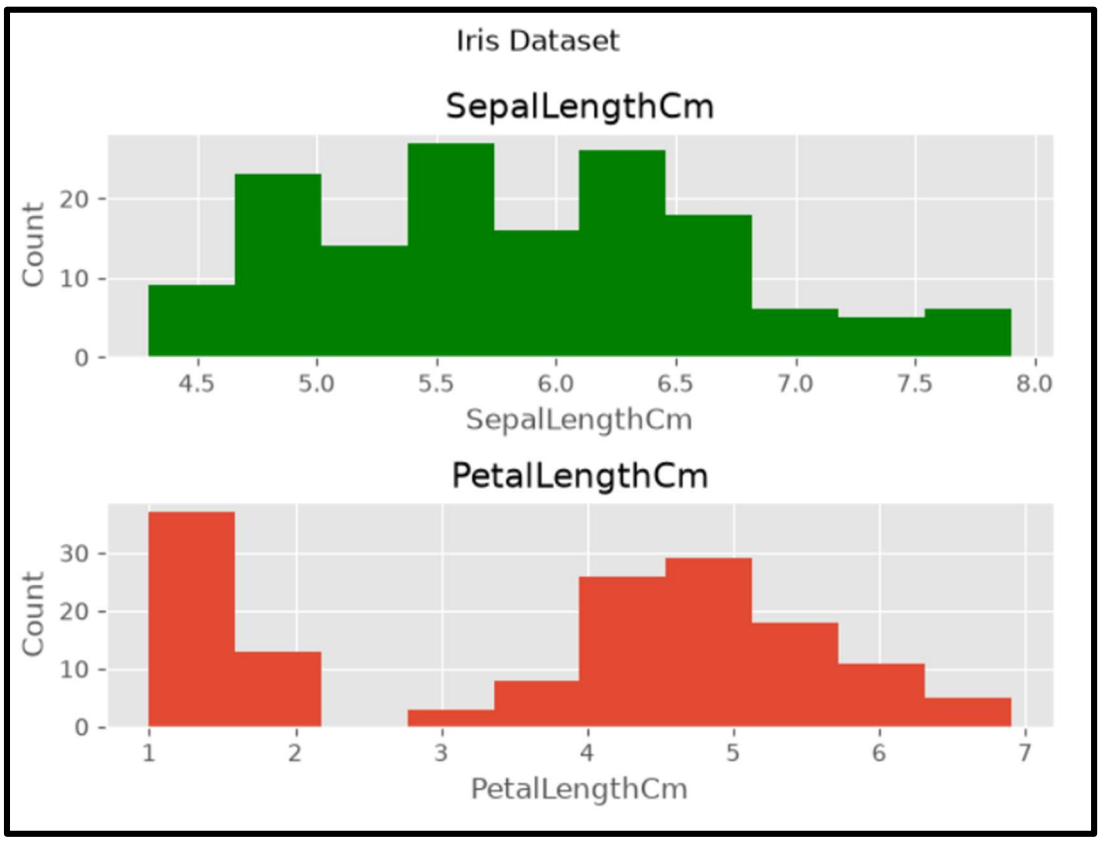
plt.hist(data.PetalLengthCm)

plt.xlabel("PetalLengthCm")

plt.ylabel("Count")

plt.tight_layout()

plt.show()
```



3. Draw Four Plots.

```
plt.suptitle("Iris Dataset")

# 1st Plot

plt.subplot(2, 2, 1)

plt.title("SepalLengthCm")

plt.hist(data.SepalLengthCm, color="green")

plt.xlabel("SepalLengthCm")

plt.ylabel("Count")
```

2nd Plot

plt.subplot(2, 2, 2)

plt.title("SepalWidthCm")

plt.hist(data.SepalWidthCm, color="blue")

plt.xlabel("SepalWidthCm")

plt.ylabel("Count")

3rd Plot

plt.subplot(2, 2, 3)

plt.title("PetalLengthCm")

plt.hist(data.PetalLengthCm, color="orange")

plt.xlabel("PetalLengthCm")

plt.ylabel("Count")

4th Plot

plt.subplot(2, 2, 4)

plt.title("PetalWidthCm")

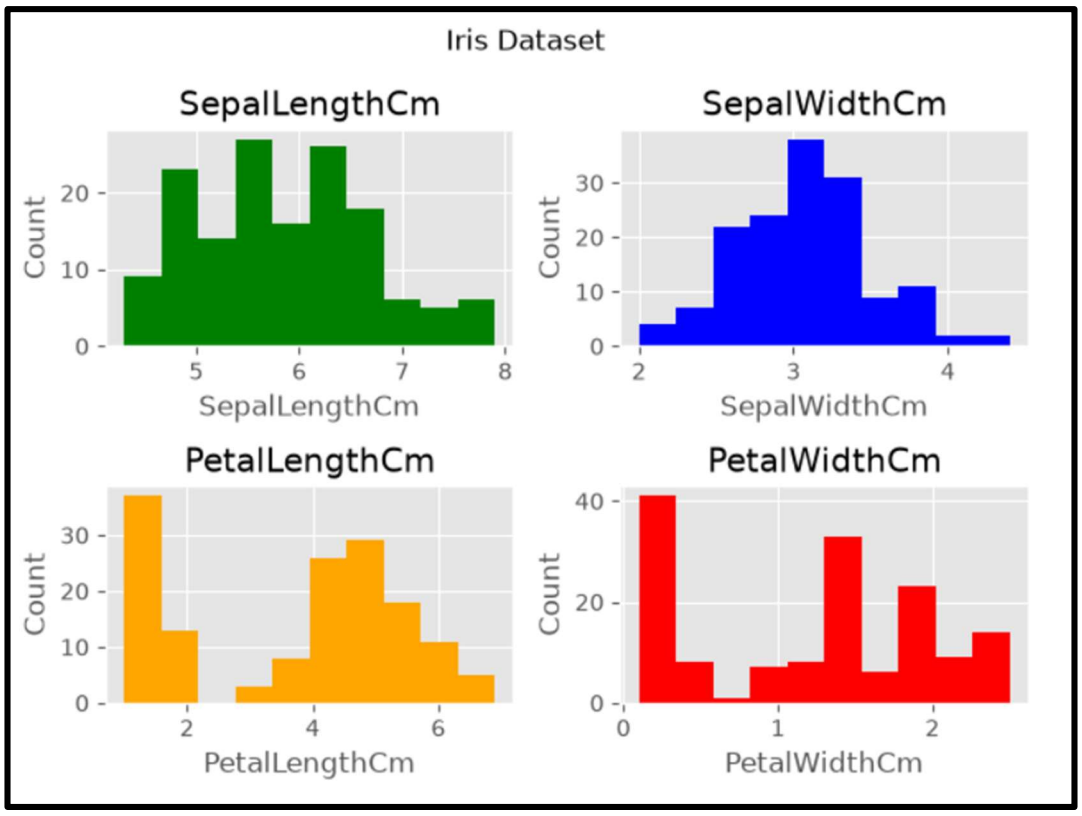
plt.hist(data.PetalWidthCm, color="red")

plt.xlabel("PetalWidthCm")

plt.ylabel("Count")

plt.tight_layout()

plt.show()



4. Change Figure Size & Color.

plt.figure(figsize=(10, 7),facecolor='lightgreen',edgecolor="black",linewidth=4)

plt.suptitle("Iris Dataset")

1st Plot

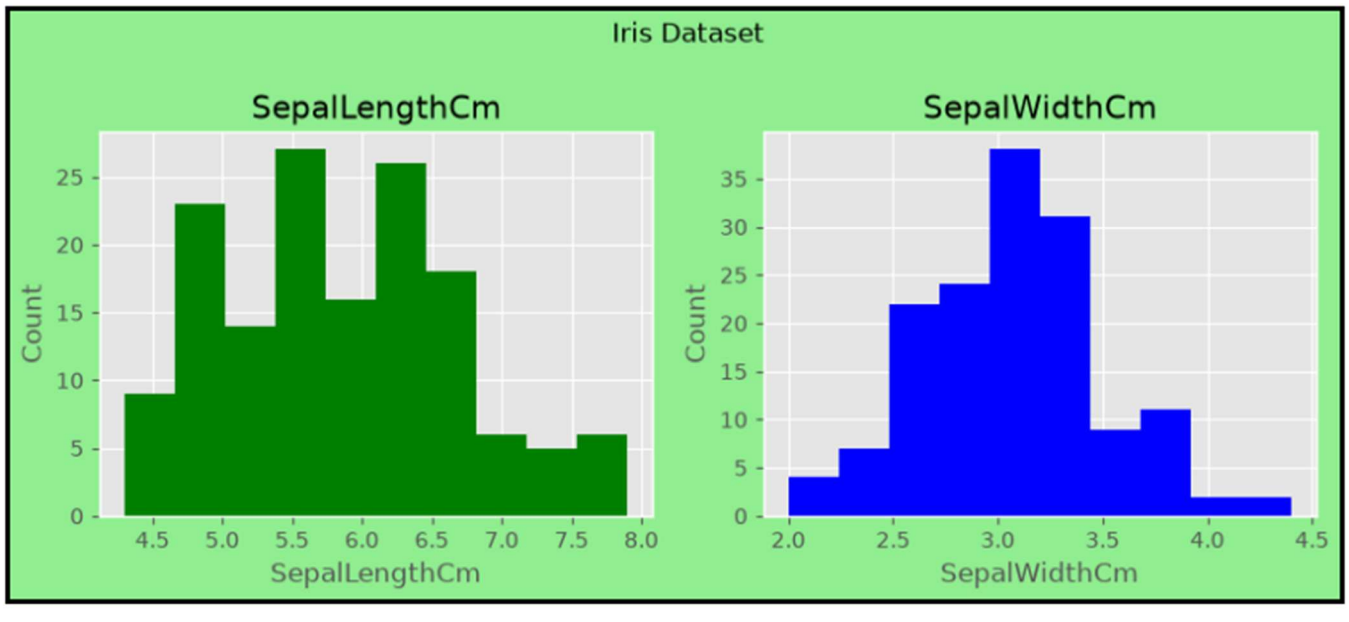
plt.subplot(2, 2, 1)

plt.title("SepalLengthCm")

plt.hist(data.SepalLengthCm, color="green")

plt.xlabel("SepalLengthCm")

plt.ylabel("Count")



```
# 2nd Plot

plt.subplot(2, 2, 2)

plt.title("SepalWidthCm")

plt.hist(data.SepalWidthCm, color="blue")

plt.xlabel("SepalWidthCm")

plt.ylabel("Count")

plt.show()
```
